5.3 CPLEX 建模

5.3.1 类与接口

(1) IloCplexModeler 类。

(2) IloModeler 接口。

(3) IloCplexs 类。

① IloCplex 类继承了 IloCplexModeler 类。

② 实现 IloCplex 类的求解方法。

③ IloCplex 类实现了 IloModeler 接口。

一般使用 IloCplex cplex = new IloCplex() 新建模型。

5.3.2 变量

1. 简介

变量类型如下。

(1) 0-1 变量：IloNumVarType.Bool。

(2) 整型变量：IloNumVarType.Int。

(3) 连续变量：IloNumVarType.Float。

变量一般使用 IloNumVar 对象。

2. 常用函数

(1) IloCplex.numVar (double lb, double ub, IloNumVarType type, String name)。

(2) IloCplex.intVar()：创建整数决策量。

(3) IloCplex.boolVar()：创建 0-1 决策量。

3. 数组形式添加

(1) numVarArray()。

(2) intVarArray()。

(3) boolVarArray()。

5.3.3 表达式

1. 简介

(1) 对于模型的目标函数和约束，基于表达式进行定义。

(2) 基于 IloNumExpr 接口及其子接口定义表达式。

(3) 方法如下。

① IloModeler.sum。

② IloModeler.prod：常用于一个表达式 (变量) 与常数的乘积。

仓库容量限制：IloModeler.prod(capacity, opened[j])。

③ IloModeler.scalProd：常用于变量数组与系数数组的点积。

仓库开通成本计算：IloModeler.scalProd(cost[c], supply[c])。

④ 仓库开通成本计算：IloModeler.diff 表示相减。

⑤ 仓库开通成本计算：IloModeler.negative 表示乘 −1。

⑥ 仓库开通成本计算：IloModeler.square。

(4) 常用的场景。

① 当表达式由多个常数项、多个系数与变量的乘积组成，采用 IloNumExpr 接口。

② 当表达式由多个系数和变量的乘积组成，采用 IloLinearNumExpr 接口。

③ IloLinearNumExpr 接口在初始化时可以指定常数部分，因此对于第一种场景也可先通过 double 类型进行求和然后在初始化的时候作为常数项一次性添加，至于剩下的多个系数与变量的乘积可使用 addTerm 方法添加。

2. 子类 IloLinearNumExpr 常用函数

常用函数如图 5.19 所示。

函数返回类型	函数表达式
void	add(IloLinearNumExpr sc)
void	addTerm(double coef, IloNumVar var)
void	addTerm(IloNumVar var, double coef)
void	addTerms(double[] coef, IloNumVar[] var)
void	addTerms(double[] coef, IloNumVar[] var, int start, int num)
void	addTerms(IloNumVar[] var, double[] coef)
void	addTerms(IloNumVar[] var, double[] coef, int start, int num)
void	clear()
double	getConstant()
IloLinearNumExprIterator	linearIterator()
void	remove(IloNumVar var)
void	remove(IloNumVar[] var)
void	remove(IloNumVar[] var, int start, int num)
void	setConstant(double val)

图 5.19 常用函数表达式

5.3.4 范围约束

(1) 使用 IloRange 对象。

(2) 表示形如 lb <= expression <= ub 的约束。

1. 添加约束

1) lb <= expression <= ub

IloRange rng = cplex.range(lb, expr, ub, name)。

(1) lb 和 ub 是 double 类型的值。

(2) expr 是 IloNumExpr 类型的实例。

(3) name 是 string 类型。

2) expression <= ub

(1) IloRange le = IloCplex.le(expr, ub, name)。

(2) IloCplex.addLe(expr, ub, name)。

(3) 相比第 1 种，第 2 种会把约束加入当前模型中。

3) expression >= lb

(1) IloRange ge = IloCplex.le(expr, lb, name)。

(2) IloCplexModeler.addGe(expr, lb, name)。

(3) 相比第 1 种，第 2 种会把约束加入当前模型中。

2. IloRange 的常用函数

常用函数如图 5.20 所示。

函数返回类型	函数表达式
void	clearExpr()
IloNumExpr	getExpr()
double	getLB()
double	getUB()
void	setBounds(double lb, double ub)
void	setExpr(IloNumExpr expr)
void	setLB(double lb)
void	setUB(double ub)

图 5.20 IloRange 的常用函数

5.3.5 目标函数

(1) 使用 IloObjective 对象。

(2) 三要素如下。

① sense：最大化或者最小化。

② expression：目标函数表达式。

③ name：目标函数名称 (可选参数)。

(3) 常用函数如下。

① IloObjective obj = IloCplexModeler.addMaximize(expr)。

② IloObjective obj = IloCplexModeler.add(cplex.maximize(expr))。

③ setExpr(expr)：可用于更新目标函数，如列生成中的定价问题。

5.3.6 建模方式

1. 按行建模

1) 参数输入 (Input Data)

foodMin[j]：食物 j 被启用的最小数量。

foodMax[j]：食物 j 被启用的最大数量。

foodCost[j]：食物 j 的单位成本。

nutrMin[i]：营养成分 i 的最小数量。

nutrMax[i]：营养成分 i 的最大数量。

nutrPerFood[i][j]：食物 j 中含有营养成分 i 的数量。

2) 决策变量 (Modeling Variables)

buy[j]：购买食物 j 的数量。

3) 目标函数 (Objective)

minimize sum(j) buy[j] * foodCost[j]。

4) 约束 (Constraints)

forall foods i:nutrMin[i]<=sum(j)buy[j]*nutrPer[i][j]<= nutrMax[j]。

按行建模代码如下。

populateByRow.java

```
1  static void buildModelByRow(IloModeler model, Data data, IloNumVar[] buy,
       IloObjective cost, IloNumVarType type)
2          throws IloException {
3      int nFoods = data.nFoods;
4      int nNutrs = data.nNutrs;
5
6      // 以循环方式新建变量
7      for (int j = 0; j < nFoods; j++) {
8          buy[j] = model.numVar(data.foodMin[j], data.foodMax[j], type);
9      }
10
```

```
    // 设置目标函数
    cost.setExpr(model.scalProd(data.foodCost, buy));

    // 按行添加约束
    for (int i = 0; i < nNutrs; i++) {
        model.addRange(data.nutrMin[i], model.scalProd(data.nutrPerFood[i],
      buy), data.nutrMax[i]);
    }
}
```

2. 按列建模

按列添加本质上是添加变量，需要考虑如下因素。

(1) 对目标函数的影响。

(2) 对约束的影响。

1) 过程

(1) 假设模型为 IloMPModeler 模型。

(2) 取出目标函数（引用），得到 IloObjective objective。

(3) 设置约束的下界和上界，得到 IloRange[] constraint。

(4) 对每个变量，执行以下操作。

① 与该变量对应的系数一起添加到目标函数中，并返回一个新的列对象 IloColumn col = model.column(objective, coefficient)。

② 对每个约束，执行以下操作。

使用 model.column(IloRange constraint[i], double val) 为变量添加在该约束中的系数，并返回一个列对象 IloColumn col2。

使用 IloColumn.and() 函数将 col 与 col2 进行“连接”，即 col = col.and(col2)。

完成对目标函数以及约束的更新。

③ 使用 model.numVar(IloColumn col, double lb, double ub, IloNum-VarType type) 为新的列设置对应的变量（包括上下界、类型）。

2) 代码示例

代码示例如下。

populateByColumn.java

```
static void buildModelByColumn(IloMPModeler model, Data data, IloNumVar[]
      buy, IloObjective cost,
        IloNumVarType type) throws IloException {
    int nFoods = data.nFoods;
    int nNutrs = data.nNutrs;

    IloRange[] constraint = new IloRange[nNutrs];
```

```
7
8       // 设置约束的下界和上界
9       for (int i = 0; i < nNutrs; i++) {
10          constraint[i] = model.addRange(data.nutrMin[i], data.nutrMax[i]);
11      }
12
13      for (int j = 0; j < nFoods; j++) {
14          // 与该变量对应的系数一起添加到目标函数中，并返回一个新的列对象
15          IloColumn col = model.column(cost, data.foodCost[j]);
16          for (int i = 0; i < nNutrs; i++) {
17              // 为变量添加在该约束中的系数，并返回一个列对象，使用IloColumn.
        //and()“连接”两列
18              col = col.and(model.column(constraint[i], data.nutrPerFood[i][j
        ]));
19          }
20          // 为新的列设置对应的变量(包括上下界、类型)
21          buy[j] = model.numVar(col, data.foodMin[j], data.foodMax[j], type);
22      }
23  }
```

3. 按非零建模

一个简单的模型（Example: LPex1.java）。

该方法是根据数据创建了如下线性规划模型：

$$\max \quad x_1 + 2x_2 + 3x_3 \tag{5.1}$$

$$\text{s.t.} \quad -x_1 + x_2 + x_3 \leqslant 20 \tag{5.2}$$

$$x_1 - 3x_2 + x_3 \leqslant 30 \tag{5.3}$$

$$0 \leqslant x_1 \leqslant 40, x_2, x_3 \geqslant 0 \tag{5.4}$$

代码示例如下。

populateByNonzero.java

```
1  static void populateByNonzero(IloMPModeler model, IloNumVar[][] var,
       IloRange[][] rng) throws IloException {
2      double[] lb = { 0.0, 0.0, 0.0 };
3      double[] ub = { 40.0, Double.MAX_VALUE, Double.MAX_VALUE };
4      IloNumVar[] x = model.numVarArray(3, lb, ub);
5      var[0] = x;
6
7      double[] objvals = { 1.0, 2.0, 3.0 };
8      model.add(model.maximize(model.scalProd(x, objvals)));
```

```
9
10      rng[0] = new IloRange[2];
11      rng[0][0] = model.addRange(-Double.MAX_VALUE, 20.0);
12      rng[0][1] = model.addRange(-Double.MAX_VALUE, 30.0);
13
14      rng[0][0].setExpr(model.sum(model.prod(-1.0, x[0]), model.prod(1.0, x
            [1]), model.prod(1.0, x[2])));
15      rng[0][1].setExpr(model.sum(model.prod(1.0, x[0]), model.prod(-3.0, x
            [1]), model.prod(1.0, x[2])));
16
17      x[0].setName("x1");
18      x[1].setName("x2");
19      x[2].setName("x3");
20      rng[0][0].setName("c1");
21      rng[0][1].setName("c2");
22  }
```

5.3.7 模型求解

(1) 调用 IloCplex.solve()。

(2) 解的类型分类 IloCplex.Status 如下。

① 有界 (Bounded)。

② 无界 (Unbounded)。

③ 可行 (Feasible)。

④ 不可行 (Infeasible)。

⑤ 不可行或无界 (InfeasibleOrUnbounded)。

⑥ 错误 (Error)。

⑦ 未知 (Unknown)。

⑧ 最优 (Optimal)。

(3) 可结合参数部分，在求解前进行相关设置。

5.3.8 获取解的信息

1) 目标函数

IloCplex.getObjValue()。

2) 变量

① 获取单个变量值 double IloCplex.getValue(var)。

② 获取变量数组的值 double[] IloCplex.getValues(vars)。

③ IloCplex.getReducedCost(ivar)。

④ IloCplex.getReducedCosts(vars)。

3）约束

（1）松弛变量如下。

① IloCplex.getSlack(IloRange rng)。

② IloCplex.getSlacks(IloRange[] rng)。

③ IloCplex.getSlacks(IloLPMatrix matrix)。

（2）对偶变量如下。

① IloCplex.getDual(IloRange rng)。

② IloCplex.getDuals(IloRange[] rng)。

③ IloCplex.getDuals(IloLPMatrix matrix)。

4）极射线

IloCplex.getRay()。

5.3.9 模型导出与模型导入

（1）模型导出。

IloCplex.exportModel("diet.lp")。

（2）模型导入。

IloCplex.importModel()。

5.3.10 参数

官方介绍参见网址https://www.ibm.com/support/knowledgecenter/zh/SSSA5P_12.8.0/ilog.odms.cplex.help/CPLEX/Parameters/topics/introListAlpha.html。

常用参数如下。

1. 运行时间

IloCplex.Param.TimeLimit。

2. Random seed

设置随机种子，IloCplex.setParam(IloCplex.Param.RandomSeed, seed)。

3. Presolve

（1）关闭预求解，IloCplex.setParam(IloCplex.Param.Preprocessing. Presolve, false)。

（2）关闭预求解缩减，IloCplex. setParam (IloCplex. Param. Preprocessing. Reduce, 0)。

4. RootAlgorithm

（1）使用单纯形算法求解第一个 LP 问题，IloCplex.setParam(IloCplex.Param. RootAlgorithm, IloCplex.Algorithm.Primal)，也可以调用如下算法。

① IloCplex.Algorithm.Auto：自动选择。

② IloCplex.Algorithm.Dual：对偶单纯形法。

③ IloCplex.Algorithm.Network：网络单纯形算法。

④ IloCplex.Algorithm.Barrier：障碍算法（内点法的一种）。

⑤ IloCplex.Algorithm.Sifting：启用 sifting 算法。

⑥ IloCplex.Algorithm.Concurrent：在多进程中启用多个算法并发。

（2）可参考示例文件 LPex7.java。

5. NodeAlgorithm

IloCplex.Param.NodeAlgorithm，可选算法如下。

① IloCplex.Algorithm.Auto：自动选择。

② IloCplex.Algorithm.Primal：单纯形法。

③ IloCplex.Algorithm.Dual：对偶单纯形法。

④ IloCplex.Algorithm.Network：网络单纯形算法。

⑤ IloCplex.Algorithm.Barrier：障碍算法（内点法的一种）。

⑥ IloCplex.Algorithm.Sifting：启用 sifting 算法。

6. Benders Strategy

IloCplex.Param.Benders.Strategy，可选策略如下。

（1）IloCplex.BendersStrategy.Auto，有两种情况。

① 情况 1：用户未向模型提供注释，此时 CPLEX 执行传统分支定界算法。

② 情况 2：用户提供了注释，此时 CPLEX 根据注释分解成主问题和子问题，并且尝试能否进一步分解子问题。

（2）IloCplex.BendersStrategy.Off：执行传统分支和界限；忽略任何 Benders 注释，也不使用 Benders 算法。

（3）IloCplex.BendersStrategy.Workers：CPLEX 根据注释确定主问题，并尝试将剩余的变量分解成不相关的子问题并交给不同的 Worker。

（4）IloCplex.BendersStrategy.Full：CPLEX 自动分解模型，忽略可能提供的任何注释。

① 将所有整数变量放入主问题。

② 将所有连续变量放入子问题。

③ 进一步分解此子问题 (如果可能)。

（5）IloCplex.BendersStrategy.User：CPLEX 严格根据注释分解模型。

注意：如果用户注释错误，CPLEX 会抛出错误，即使 Auto 也不是完全“智能”的。

Benders 注释分解示例如下。

Benders comment.java

```
1 // Create an annotation and you can set a default value here.
```

```
IloCplex.LongAnnotation benders = cplex.newLongAnnotation(IloCplex.CPX_
        BENDERS_ANNOTATION);

// Put the binary "use" variables in the master problem, 0 represents master problem.
for (IloNumVar u : use) {
    cplex.setAnnotation(benders, u, 0);
}

/*
 * The LP portion does not decompose into smaller problems,
 * so we put all the "ship" variables in subproblem 1.
 * If the LP portion can be decomposed, wen can assign 2, 3, 4... to the variables.
 */
for (IloNumVar[] s : ship) {
    for (IloNumVar s0 : s) {
        cplex.setAnnotation(benders, s0, 1);
    }
}
// Set the default Benders strategy to be adherence to our design.
cplex.setParam(IloCplex.Param.Benders.Strategy,
                    IloCplex.BendersStrategy.User);
```

7. Search Strategy

IloCplex.Param.MIP.Strategy.Search，可选策略如下。

（1）IloCplex.MIPSearch.Auto 默认自动选择。

（2）IloCplex.MIPSearch.Traditional 使用分支定界策略。

（3）IloCplex.MIPSearch.Dynamic 使用动态搜索策略。

8. MIP 割平面开关

1）Gomory 割（Gomory Cuts）

IloCplex.Param.MIP.Cuts.Gomory。

2）最大团割（Maximal Cliques Cuts）

（1）IloCplex.Param.MIP.Cuts.Cliques。

（2）如果一组 0-1 变量满足任意可行解，至多只有一个变量取值为正，那么这种关系叫作团。在优化之前，CPLEX 首先会构建一个图表示这些关系，并且找到该图中的最大团。

3）最小覆盖割（Minimal Cover Cuts）

（1）IloCplex.Param.MIP.Cuts.Covers。

（2）如果一个约束是背包形式的约束 (即一系列 0-1 变量通过与非负系数相乘再相加，且小于或等于右端项)，那么该约束对应一个最小覆盖。一个最小覆盖指的是不等式包含的

变量的子集满足如下条件：如果子集中的决策变量取值均为 1，则该背包约束会被违背；但是，如果排除一个子集，则约束会被满足。CPLEX 会针对此条件产生一个约束，并且该割平面被称为覆盖割。

4）MIR 割（MIR Cuts）

（1）IloCplex.Param.MIP.Cuts.MIRCut。

（2）MIR 割平面是通过对决策变量的约束系数和约束的右端项进行整数圆整而生成的。

5）零半割（Zero-half Cuts）

（1）IloCplex.Param.MIP.Cuts.ZeroHalfCut。

（2）零半割基于一个不等式由整数变量和整数约束系数组成，因此可以对右端项进行向下取整。

6）流覆盖割（Flow Cover Cuts）

（1）IloCplex.Param.MIP.Cuts.FlowCovers。

（2）流覆盖是根据包含连续变量的约束生成的。要使连续变量的变量上限为零或正数，取决于相关的 0-1 变量的设置。流覆盖的想法来自将包含连续变量的约束定义为网络中的单个节点，其中连续变量分为流入和流出。流量打开或关闭，取决于 0-1 变量上界的相关设置。单节点上的流量和需求实际上相当于背包约束。因此，可以使用背包约束生成流覆盖割（即根据连续变量及其变量的上界来生成流覆盖割）。

7）流路径割（Flow Path Cuts）

（1）IloCplex.Param.MIP.Cuts.PathCut。

（2）流路径割是通过考虑一组约束条件生成的，通过一些连续变量定义网络中路径结构。其中，约束代表节点；连续变量是流入和流出；流的开启或关闭取决于相关 0-1 变量的取值。

8）全局隐界割（Global Implied Bounds Cuts）

（1）IloCplex.Param.MIP.Cuts.Implied。

（2）在一些模型中，0-1 变量会对非 0-1 变量（即一般整型变量和连续型变量）的界限起到暗示作用。CPLEX 会根据这些关系产生相应的割平面。

（3）CPLEX 将会利用全局的有效界限，针对模型中的连续型变量产生全局隐界割。在 CPLEX 中，也有控制该行为的参数。具体见 CPLEX 用户手册中的参数部分。

9）提升与投影割（Lift and Project Cuts）

（1）IloCplex.Param.MIP.Cuts.LiftProj。

（2）参见网址 https://www.ibm.com/support/knowledgecenter/en/SSSA5P_12.6.3/ilog.odms.cplex.help/CPLEX/UsrMan/topics/discr_optim/mip/cuts/35_liftproj.html。

9. MIP 分支方向

IloCplex.Param.MIP.Strategy.Branch，分支方向可选如下。

（1）IloCplex.BranchDirection.Auto。

(2) IloCplex.BranchDirection.Down。

(3) IloCplex.BranchDirection.Up。

(4)一般结合 makeBranch(IloNumVar var, double bound, BranchDirection dir, double objestimate) 使用。

10. MIP 解池中的目标值允许 Gap

IloCplex.Param.MIP.Pool.RelGap，用于判断是否保留解到解池中，例如，如果将此参数设置为 0.01，那么将丢弃与目标值的差大于或等于 1% 的解。

11. MIP 节点数量限制

IloCplex.Param.MIP.Limits.Nodes，用于设置在算法终止而未达到最优性之前求解的最大节点数。

12. MIP 强调参数

IloCplex.Param.Emphasis.MIP，用于控制 MIP 中速度、可行性、最优性和移动界限之间的折衷，可选参数如下。

(1) 0：在使用默认设置 BALANCED 时，CPLEX 致力于快速证明最优性，但要在快速证明最优性和在优化早期找到高质量可行解法之间作均衡。

(2) 1：在将此参数设置为 FEASIBILITY 时，CPLEX 将在优化问题时频繁生成更多可行的解法，牺牲一部分最优性证明速度。

(3) 2：在设置为 OPTIMALITY 时，早期阶段用于查找可行的解法的工作量较少。

(4) 3：使用设置 BESTBOUND 时，将通过移动最佳界限值来更着重强调证明最优性，因此顺便检测可行的解法几乎成为偶然。

(5) 4：在参数设置为 HIDDENFEAS 时，MIP 优化器更努力查找非常难于找到的高质量可行的解法，因此在 FEASIBILITY 设置难以找到可接受质量的解法时考虑此设置。

13. MIP 节点日志显示信息

IloCplex.Param.MIP.Display，控制在混合整数优化（MIP）期间 CPLEX 报告到屏幕并记录在日志中的内容，可选信息如下。

(1) 0：找到最佳解法之后才显示。

(2) 1：显示整数可行解。

(3) 2：默认值，以 MIP 节点日志区间设置的频率显示整数可行解法以及条目。

(4) 3：显示自先前显示以来添加的割平面数；关于每个成功的 MIP 开始的处理的信息；整数可行解法的耗用时间（以秒为单位）和耗用时间（以确定性标记为单位）。

(5) 4：显示从先前选项提供的信息以及关于根处的 LP 子问题的信息。

(6) 5：显示从先前选项提供的信息以及关于根和节点处的 LP 子问题的信息。

14. MIP 节点日志间隔

IloCplex.Param.MIP.Interval，可选值如下。

（1）$n<0$：显示新当前项，并在求解开始时频繁显示日志行，在求解进行时显示的频率降低。

（2）0：让 CPLEX 决定记录节点的频率（缺省值）。

（3）$n>0$：显示新当前项，并每隔 n 个节点显示一行日志。

15. MIP 启发式频率

IloCplex.Param.MIP.Strategy.HeuristicFreq，用于控制使用启发式算法的频率，可选值如下。

（1）−1：关闭启发式算法。

（2）0：自动：让 CPLEX 选择；且为默认选项。

（3）正整数：以该频率周期性地使用启发式算法。

5.3.11 其他

逻辑约束如下。

（1）IloCplexModeler.and。

（2）IloCplexModeler.or。

（3）IloCplexModeler.not。

（4）IloCplexModeler.ifThen。

例如:IloCplexModeler.ifThen (cplex.eq (varUsed[i],0), cplex.eq (buy[i], 0.0))。

（5）IloCplexModeler.min。

（6）IloCplexModeler.max。

5.4 传统回调

IloCplex.Callback 类定义了支持所有 Callback 操作的接口。Callback 对象需要用户实现 main 方法，该 main 方法在 CLPEX 优化过程的相应“节点”被自动调用。要实现 Callback 需遵循以下原则。

（1）选择并继承相应的 Callback 类。

（2）实现 main 方法，自定义的 main 方法可以使用父类的方法。

（3）如果默认的 clone 函数功能不全面，并且回调函数用于并行优化，那么该函数还需要用户自己实现。默认的 clone 函数执行的是表的复制，因此通常用户需要在同一线程中执行对象的深拷贝，或在多线程中，当需要同步化（synchronization）时，必须使用关键词 synchronize。

（4）新建自定义 Callback 类的实例，使用 IloCplex.use(callback 实例）添加，之后 IloCplex object 进行求解时，会在相应的“节点”调用用户定义的 callback 方法。可能会用到 (IloLPMatrix)cplex.LPMatrixIterator().next() 方法，其中 CPLEX 为 IloCplex 实例。

① IloCplex.use。

② IloCplex.setParam。

③ IloCplex.solve。

④ IloCplex.end。

（5）main 方法是 Protected 类，是为了确保该方法只被用于产生用户定义的回调函数，或者去实现其中的主函数。

5.4.1 参考回调

1. 简介

参考回调（Informational callback）是用户编写的例程，具有如下特性。

（1）使应用程序能够访问关于当前混合整数规划（MIP）优化的信息而不牺牲性能。

（2）不会干扰解空间的搜索。

（3）参考回调还可以使应用程序能够终止优化。

（4）与 MIP 动态搜索兼容。对于许多模型，MIP 动态搜索可以比常规 MIP 算法的分支和剪枝更快，因此可以更快找到可行解和最优解。

（5）与并行优化的所有方式兼容。

（6）CPLEX 确保以线程安全方式并以确定性顺序调用参考回调。

2. 涉及的类

涉及的类为 ilog.cplex.IloCplex.MIPInfoCallback（可查看可返回的信息）。子类如下。

① IloCplex.DisjunctiveCutInfoCallback。

② IloCplex.FlowMIRCutInfoCallback。

③ IloCplex.FractionalCutInfoCallback。

④ IloCplex.ProbingInfoCallback。

3. 示例文件 MIPex4.java

1）功能一：基于求解时间和 gap 控制程序是否终止

（1）getMIPRelativeGap()。

（2）getCplexTime()。

2）功能二：打印日志

（1）搜寻到了更好的解。

① hasIncumbent()。

② getIncumbentObjValue()。

③ getIncumbentValues()。

(2) 每隔 100 个 Node 打印一次。

getNremainingNodes64()。

5.4.2 查询或诊断回调

1. 简介

查询回调或诊断回调（Query or Diagnostic callback）可以监视正在进行的优化，具有如下特点。

(1) 可选择性地终止程序。

(2) 与执行参考回调相比，查询回调访问关于当前优化的更详细信息。副作用是，可能会减慢进度。

(3) 与动态搜索不兼容。

(4) 就常规分支切割树的横越进行假设；在动态搜索期间或者在并行优化中的确定性搜索期间，关于混合整数规划（MIP）的这些假设可能出现错误。

2. 调用位置

(1) 在从内点解到单纯形法基底的转换期间，将定期调用转换查询回调。

IloCplex.CrossoverCallback。

(2) 在网络单纯形法算法期间，将定期调用网络查询回调。

IloCplex.NetworkCallback。

(3) 在内点法算法期间的每次迭代中，将调用内点查询回调。

① IloCplex.BarrierCallback。

② IloCplex.ContinuousCallback。

(4) 在单纯形法算法期间的每次迭代中，将调用单纯形法查询回调。

① IloCplex.SimplexCallback。

② IloCplex.ContinuousCallback。

(5) 在分支切割搜索期间，将定期调用 MIP 查询回调。

IloCplex.MIPCallback。

(6) 在探测期间，将定期调用探测查询回调。

IloCplex.ProbingCallback。

(7) 在生成小数割平面期间，将定期调用小数割平面查询回调。

IloCplex.FractionalCutCallback。

(8) 在生成分离式割平面期间，将定期调用分离式割平面查询回调。

IloCplex.DisjunctiveCutCallback。

(9) 在生成流和混合整数舍入（MIR）割平面期间，将定期调用流和 MIR 割平面查询回调。

IloCplex.FlowMIRCutCallback。

5.4.3 控制回调

1. 简介

控制回调允许您控制 MIP 问题优化期间执行的分支切割搜索。具有如下特点。

(1) 控制回调会干预搜索，会使得 CPLEX 关闭动态搜索。

(2) 缺省情况下（CPLEX 不以并行方式运行），并行 MIP 求解也处于关闭状态。

(3) 可以通过将线程参数设置为非零正值来启用并行求解，但必须特别注意采用回调函数才能实现确定性搜索过程。

2. 调用位置

1) 节点回调

(1) IloCplex.NodeCallback。

(2) 在分支切割搜索期间查询 CPLEX 将要处理的下一个节点并选择性地加以覆盖。

2) 求解回调

(1) IloCplex.SolveCallback。

(2) 指定和配置用于在各个节点处求解 LP 的优化器选项。

(3) 特有函数，如图 5.21 所示。

函数返回类型	函数表达式
protected IloCplex.CplexStatus	getCplexStatus()
protected IloCplex.Status	getStatus()
boolean	isDualFeasible()
boolean	isPrimalFeasible()
protected void	setStart(double[] x, IloNumVar[] var, double[] pi, IloRange[] rng)
protected void	setStart(double[] x, IloNumVar[] var, int xstart, int xnum, double[] pi, IloRange[] rng, int cstart, int cnum)
protected boolean	solve()
protected boolean	solve(int alg)
protected void	useSolution()

图 5.21 特有函数

3) 用户割平面回调

(1) IloCplex.UserCutCallback。

(2) 在每个节点处添加针对特定问题的用户定义割平面。

(3) 当希望 CPLEX 的所有割平面添加完毕之后再添加自定义的 Cut，判断 isAfterCutLoop() 为 Ture，再执行代码即可。

(4) 特有函数。

用户割平面回调的相关函数信息如图 5.22 所示。

函数返回类型	函数表达式
protected void	abortCutLoop()
protected IloRange	add(IloRange cut, int cutmanagement)
protected IloRange	addLocal(IloRange cut)
protected boolean	isAfterCutLoop()

图 5.22 用户割平面回调

4）惰性约束回调

（1）IloCplex.LazyConstraintCallback。

（2）添加惰性约束（除非违反，否则不会进行评估的约束）。

（3）特有函数。

惰性约束回调的相关函数信息如图 5.23 所示。

函数返回类型	函数表达式
protected IloRange	add(IloRange cut)
protected IloRange	add(IloRange cut, int cutmanagement)
protected IloRange	addLocal(IloRange cut)
protected int	getSolutionSource()
protected boolean	isUnboundedNode()

图 5.23 惰性约束回调

5）探试回调

（1）IloCplex.HeuristicCallback。

（2）实现探试，该探试将尝试根据每个节点处的 LP 松弛解生成新的现任解。

6）分支回调

（1）IloCplex.BranchCallback。

（2）查询 CPLEX 在每个节点处进行分支的方式并选择性地加以覆盖。

7）现任解回调

（1）IloCplex.IncumbentCallback。

（2）检查 CPLEX 在搜索期间找到的现任解并选择性地予以拒绝。

5.4.4 回调的终止

将类 IloCplex.Aborter 的实例传递给 IloCplex 的实例。然后，调用 IloCplex.Aborter.abort 方法以终止优化。

5.4.5 传统回调示例

1. AdMIPex1: node and branch callbacks

（1）继承 IloCplex.BranchCallback 生成自己的分支类，并设置属性及构造方法 IloNumVar[]_vars。

（2）实现 main 方法，可能用到的方法如下。

① 判断分支策略：getBranchType() 和 IloCplex.BranchType.BranchOnVariable。

② 选择分支变量：getFeasibilities() 和 IloCplex.IntegerFeasibilityStatus.Infeasibles。

③ 构造分支：makeBranch() 和 IloCplex.BranchDirection.Up/Down。

④ 调用自定义的 BranchCallback 子类。

（3）node callbacks。

① 继承 IloCplex.NodeCallback 生成自己的分支类。

② 实现 main 方法，可能用到的方法：getInfeasibilitySum() 和 selectNode()。

③ 调用自定义的 NodeCallback 子类。

2. AdMIPex3: Special Ordered Sets Type 1

（1）继承 IloCplex.BranchCallback 生成自己的分支类，并设置属性（IloSOS1）及构造方法 IloSOS1[] _sos。

（2）实现 main 方法，可能用到的方法如下。

① 选择分支变量：getFeasibilities()、IloCplex.IntegerFeasibilityStatus.Infeasible 和 getNumVars。

② 构造分支：makeBranch() 和 IloCplex.BranchDirection。

（3）调用自定义的 BranchCallback 子类。

3. AdMIPex2: HeuristicCallback

（1）继承 IloCplex.HeuristicCallback 生成自己的启发式类，并设置属性及构造方法 IloNumVar[]_vars。

（2）实现 main 方法，可能用到的方法如下。

① 变量处理：getFeasibilities() 和 IloCplex.IntegerFeasibilityStatus.Infeasible。

② 构造新的解：setSolution()。

（3）调用自定义的 HeuristicCallback 子类,可能会用到的方法:IloCplex.getNSOS1() 和 IloCplex.SOS1iterator()。

4. AdMIPex4: UserCutCallback and LazyConstraintCallback

（1）直接定义 makeCuts 方法，生成新的割约束，返回 IloRange 对象。

（2）添加 Cuts，方法为 cplex.addUserCuts() 和 cplex.addLazyConstraints()。

5. AdMIPex5: UserCutCallback and LazyConstraintCallback

1）UserCutCallback

将线性松弛中的一些可能会被违背的约束动态地分离出去。

（1）继承 IloCplex.UserCutCallback 生成自己的 UserCut 类，并设置属性及构造方法为 IloModeler modeler 和 IloNumVar[]_vars。

（2）实现 main 方法，可能用到的方法为添加 Cut，可采用 add() 和 IloCplex.CutManagement.UseCutPurge(如果这个 Cut 是无效的，CPLEX 可能会将其清除)。

（3）调用自定义的 UserCutCallback 子类。

2）UserCutCallback

在算法的每次迭代中都会被查看的 Cuts。

（1）继承 IloCplex.UserCutCallback 生成自己的 UserCut 类。

（2）实现 main 方法，可能用到的方法为添加 Cut，可采用 (IloRange)cut.getLB() 和 (IloRange)cut.getUB()。

（3）调用自定义的 UserCutCallback 子类。

3）LazyConstraintCallback

（1）继承 IloCplex.LazyConstraintCallback 生成自己的 LazyCut 类，采用方法为 IloModeler modeler 和 IloNumVar[]_vars。

（2）实现 main 方法，可能用到的方法如下。

① 约束处理：IloLinearNumExpr sum = modeler.linearNumExpr()、sum.addTerm(1.0, supply[c][j]) 和 IloCplex.IntegerFeasibilityStatus.Infeasible。

② 添加 Cut：add(modeler.le(sum, 0.0))。

（3）调用自定义的 LazyConstraintCallback 子类。

6. AdMIPex6: SolveCallback：在根节点给模型传递一个解

（1）继承 IloCplex.SolveCallback 生成自己的启发式类，并设置属性及构造方法 IloNumVar[]_vars。

（2）实现 main 方法，可能用到的方法为设置解，可采用 setStart()。

（3）调用自定义的 SolveCallback 子类，可能会用到的方法如下。

① IloConversion relax = IloCplex.conversion(lp.getNumVars(), IloNumVarType.Float)。

② cplex.add(relax)。

③ cplex.delete(relax)。

5.5 通用回调

5.5.1 简介

通用(Generic)回调是搜索解期间在许多不同位置调用的回调。基于 IloCplex.Callback.Function 接口实现（需要覆盖 invoke 方法）具有如下特点。

（1）与动态搜索兼容。

（2）无须禁用任何 MIP 功能。

（3）监视进度以及引导搜索。

（4）几乎专门处理原始模型；即，除某些情况以外，无权访问预求解模型和有关预求解模型的可用信息。

（5）不会隐式更改 CPLEX 使用的线程数。

（6）不会隐式序列化多线程求解中回调的执行。

（7）提供比旧回调更高的灵活性，可在调用时根据回调的上下文（Context）定义相应的操作。

5.5.2 功能

（1）查询有关当前求解状态和进度的信息。

（2）插入探试解。

（3）插入整数可行解作为候选值。

（4）获取当前松弛解。

（5）添加用户割平面。

（6）强制终止。

注意：通用回调暂时不支持以下功能。

（1）自定义的分支，节点选择，在节点处由用户提供的解。

（2）不能与传统回调混用。

（3）不适用于连续型问题。

（4）需要用户确保线程安全。

5.5.3 通用回调的上下文

CPLEX 调用通用回调时所处的上下文确定可从该调用回调合理执行的操作。

1. IloCplex.Callback.Context.Id 中的上下文常量

1）线程开启（ThreadUp）

当 CPLEX 开启一个线程后，该常数用于指定 CPLEX 的通用回调。

2）线程关闭（ThreadDown）

当 CPLEX 关闭一个线程后，该常数用于指定 CPLEX 的通用回调。

3）本地进程（LocalProgress）

（1）CPLEX 在进行线程本地进度时，会在此上下文中调用通用回调。

（2）线程局部进度是在 CPLEX 使用的线程之上发生的进度，但尚未提交给全局解的结构。从该上下文中调用的回调查询的信息仅在调用线程中有效。

（3）当前解的进程的信息可以通过 getIntInfo()、getLongInfo() 和 getIncumbent() 方法获得。

4）全局进程（GlobalProgress）

（1）当 CPLEX 开启全局进行之后（也就是说，当新的信息被提交给全局解的结构时），它将会启用通用回调。

（2）当前解的进程的全局有效信息可以通过 getIntInfo()、getLongInfo() 和 getIncumbent() 方法获得。

5）候选解（Candidate）

当 CPLEX 找到整数可行解的新候选解或遇到无界的线性松弛模型时，CPLEX 会调用通用回调。CPLEX 为回调提供了拒绝候选解或无界的线性松弛模型的机会。

（1）如果 isCandidatePoint() 函数表明 CPLEX 已经找到了一个候选可行点，那么可以通过调用 getCandidatePoint() 函数查询该点。

（2）如果 isCandidateRay() 函数表明 CPLEX 遇到一个无界的线性松弛模型，那么可以通过调用 getCandidateRay() 函数查询该无界射线。

（3）无论哪种情况，用户都可以通过函数 rejectCandidate() 拒绝可行的点或无界的方向。

（4）使用参数 candidate 有如下两种不同的方法。

① 作为传递给通用回调函数的值，用于指定在哪个上下文中调用通用回调。

② 与函数 CPXXcallbacksetfunc 的 where 参数进行按位“或”运算和函数 CPXcallbacksetfunc 指定在什么情况下 CPLEX 应该调用通用回调。

6）松弛（Relaxation）

（1）当发现可用的松弛解时，CPLEX 会在此上下文中调用通用回调。松弛解通常不是整数可行解。

（2）可以通过调用 getRelaxationPoint() 函数查询松弛解。

2. 使用方法

1）定义“上下文掩码”

（1）`long contextMask = IloCplex.Callback.Context.Id.Relaxation`。

（2）可以进行按位“或”操作：`contextMask |= IloCplex.Callback.Context.Id.Candidate`。

2）`IloCplex.use(IloCplex.Callback.Function, contextMask)`

3）常用方法总结

常用方法总结如图 5.24 所示。

	getID()	获得回调的上下文
	inThreadUp() **inThreadDown()** **inLocalProgress()** **inGlobalProgress()** **inCandidate()** **inRelaxation()**	验证回调函数是否已经被一个特定的上下文调用
	postHeuristicSolution (*variables*, *valeurs*, *objective*, *strategic*)	为CPLEX提供一个新的整数解
INTEGER	**isCandidatePoint()** **isCandidateRay()**	测试回调函数是否在整数(点)或者无界(射线)解处被调用
INTEGER	**getCandidatePoint**(*variables*, *valeurs*) **getCandidatePoint**(*variable*)	获得当前整数解对应的变量取值
INTEGER	**getCandidateValue**(*expression*)	获得表达式的当前值
INTEGER	**getCandidateObjective()**	获得当前的目标函数值
INTEGER	**rejectCandidate**(*contraintes*) **rejectCandidate**(*contrainte*= *0*)	通过添加约束(惰性约束)拒绝当前整数解。如果当前整数解为NULL，则CPLEX会自动添加割平面
RELAXED	**getRelaxationPoint**(*variables*, *valeurs*) **getRelaxationPoint**(*variable*)	获得当前松弛解的值
RELAXED	**getRelaxationValue**(*expression*)	获得表达式的当前值
RELAXED	**getRelaxationObjective()**	获得当前的目标函数值
RELAXED	**addUserCut**(*contrainte*, *cutManagementFlag*, *localFlag*)	添加一个局部或者全局用户自定义割平面
	getIncumbent(*variable*) **getIncumbent**(*variables*, *valeurs*)	获得当前最优可行解的值
	getIncumbentValue(*expression*)	获得当前最优可行解表达式的值
	getIncumbentObjective()	获得当前最优可行解的目标值

图 5.24 常用方法总结

5.5.4 通用回调示例

1. AdMIPex8: Cut

(1) 继承 `IloCplex.Callback.Function` 接口生成通用回调类，并设置属性及构造方法。

(2) `contextMask = IloCplex.Callback.Context.Id.Relaxation | IloCplex.Callback.Context.Id.Candidate`。

(3) 分别实现几种不同场景的对应函数。

① `disaggregate()`。

针对 `Id = Relaxation` 的情况。

目的：添加分解约束，将客户和设施位置的决策耦合起来。

`context.addUserCut()` 添加 Cut，使用 `IloCplex.CutManagement.UseCutPurge`。

② `cutsFromTable()` 函数。

针对 Id = Relaxation 的情况。

目的：用一个 cuts 的静态表来查看被违背的 cuts。

context.addUserCut() 添加 Cut，使用 IloCplex.CutManagement.UseCutPurge。

③ lazyCapacity()。

针对 Id = Candidate 的情况。

借助 isCandidatePoint() 判断。

添加约束。

context.rejectCandidate()。

（4）覆盖 invoke 方法，根据 context 值采取相应的操作。

（5）使用 IloCplex.use(Function callback, long contextMask) 将通用回调添加到 CPLEX 中。

2. AdMIPex9: HeuristicCallback

（1）继承 IloCplex.Callback.Function 接口生成通用回调类，并设置属性及构造方法。

（2）contextMask = IloCplex.Callback.Context.Id.Relaxation | IloCplex.Callback.Context.Id.Candidate。

（3）启发式算法函数设计，这里用 roundDown 取整示意。采用 context.postHeuristicSolution() 添加启发式解到队列中。

（4）覆盖 invoke 方法，根据 context 值采取相应的操作。

（5）添加通用回调到 CPLEX 中。

① 关闭 CPLEX 的启发式算法，cplex.setParam(IloCplex.Param.MIP. Strategy.HeuristicFreq, -1)。

② 使用 IloCplex.use(Function callback, long contextMask) 将通用回调添加到 CPLEX 中。

5.6 Java 调用 CPLEX 求解整数规划的小例子

5.6.1 书架生产问题

一个图书馆需要搭建一些书架来放置 200 本 4 英寸（1 英寸 =2.54 厘米）高的书籍、100 本 8 英寸高的书籍和 80 本 12 英寸高的书籍，每本书厚 0.5 英寸。图书馆有下面几种摆放书籍的方式：第 1 种，可以搭建一个 8 英寸高的架子，以存储所有高度小于或等于 8 英寸的书，并且可以为 12 英寸高的书建立一个 12 英寸的高架子；第 2 种，可以搭建一个 12 英寸高的架子存储所有书籍。已知每搭建一个书架的成本为 2300 美

元，存放每平方英寸的书籍需要花费 5 美元的成本。（假设存储书籍所需的面积等于存储区域的高度 × 书的厚度。）

请建立模型并求解该问题，以帮助图书馆确定如何搭建书架才能以最小的成本放置所有书籍。

我们引入下面的决策变量。

y_i：0-1 变量；如果搭建第 i 种书架，$y_i = 1$，否则，$y_i = 0$。

x_{ij}：整数变量；在第 j 个书架上放置第 i 种书的数量。

其中 $i \in B$，$j \in S$，$S = \{1,2,3\} = B$，B 是书的种类的集合，S 是书架的种类的集合。

另外，我们引入下面的参数。

c_i：第 i 种书籍放置在一个书架上的存储成本，$\forall i \in S$。

即 $c_1 = (4 \times 0.5) \times 5 = 10$ 美元，$c_2 = (8 \times 0.5) \times 5 = 20$ 美元，$c_3 = (12 \times 0.5) \times 5 = 30$ 美元。

根据上面的信息，我们给出该问题的模型如下：

$$\min \quad \sum_{i \in B} \sum_{j \in S} x_{ij} c_i + 2300 \sum_{j \in S} y_j \tag{5.5}$$

$$\text{s.t.} \quad \sum_{j \in S} x_{1j} = 200 \tag{5.6}$$

$$\sum_{j \in S} x_{2j} = 100 \tag{5.7}$$

$$\sum_{j \in S} x_{3j} = 80 \tag{5.8}$$

$$\sum_{i \in B} x_{ij} - M y_j \leqslant 0, \qquad \forall j \in S \tag{5.9}$$

$$y_j \in \{0,1\}, x_{ij} \in \mathbb{Z}, \forall i \in B, \forall j \in S \tag{5.10}$$

其中，M 是一个很大的正数。

我们用 Java 调用 CPLEX 来求解上面的问题。代码如下：

CPLEX Java Interface

```
1 package Example;
2
3 import ilog.concert.*;
4 import ilog.cplex.*;
5 import java.util.*;
6
7 public class Main {
8
```

```
    public static void main(String[] args) throws IOException,
IloException {

        double[] cost = {10, 20, 30};
        IloCplex Cplex = new IloCplex();

        IloNumVar[][] X = new IloNumVar[3][3];
        IloNumVar[] Y = new IloNumVar[3];
        for(int i = 0; i < 3; i++){
            Y[i] = Cplex.boolVar("Y" + i);
            for(int j = 0; j < 3; j++){
                X[i][j] = Cplex.intVar(0, 200, "X" + i + j);
            }
        }

        IloNumExpr obj = Cplex.numExpr();
        for (int 0 = 0; i < 3; i++) {
            obj = Cplex.sum(obj, Cplex.prod(2300, Y[i]));
            for (int j = 0; j < 3; j++) {
                obj = Cplex.sum(obj, Cplex.prod(cost[i],
X[i][j]));
            }
        }

        Cplex.addMinimize(obj);


        //Linear expression
        IloNumExpr expr = Cplex.numExpr();
        //Constraint 1:
        expr = Cplex.numExpr();
        for(int j = 0; j < 3; j++){
            expr = Cplex.sum(expr, X[0][j]);
        }
        Cplex.addEq(expr, 200);

        //Constraint 2:
        expr = Cplex.numExpr();
        for(int j = 0; j < 3; j++){
            expr = Cplex.sum(expr, X[1][j]);
        }
        Cplex.addEq(expr, 100);
```

```
49
50                  //Constraint 3:
51                  expr = Cplex.numExpr();
52                  for(int j = 0; j < 3; j++){
53                          expr = Cplex.sum(expr, X[2][j]);
54                  }
55                  Cplex.addEq(expr, 80);
56
57                  //Constraint 4:
58                  for(int j = 0; j < 3; j++){
59                          expr = Cplex.numExpr();
60                          expr = Cplex.sum(expr, Cplex.prod(-100000, Y[j]));
61                          for(int i = 0; i < 3; i++) {
62                                  expr = Cplex.sum(expr, X[i][j]);
63                          }
64                          Cplex.addLe(expr, 0);
65                  }
66
67                  //Solve the model
68                  Cplex.solve();
69
70
71                  //------------------Solution--------------------------
72                  System.out.println("\n\n----------Solution is ---------\n");
73                  System.out.println("Objective\t\t:" + Cplex.getObjValue());
74                  for (int i = 0; i < 3; i++) {
75                          for (int j = 0; j < 3; j++) {
76                                  if(Cplex.getValue(X[i][j]) > 0) {
77                                          System.out.println("x[" + i + "," +
        j + "] = " + Cplex.getValue(X[i][j]) );
78                                  }
79                          }
80                  }
81
82                  for (int i = 0; i < 3; i++) {
83                          if(Cplex.getValue(Y[i]) > 0) {
84                                  System.out.println("y[" + i + "] = " + Cplex
        .getValue(Y[i]));
85                          }
86                  }
87          }
88  }
```

运行结果如下：

Results

```
----------The solution----------

Objective                :8700.0
x[0,2] = 200.0
x[1,2] = 100.0
x[2,2] = 80.0
y[2] = 1.0
```

根据结果，我们知道，图书馆应该搭建一个 12 英寸高规格的书架，总成本为 8700 美元。

5.6.2 包装盒选择问题

一家公司销售 7 种类型的包装盒，体积从 17 到 33 立方英尺不等。下面的需求表列出了每个盒子的体积。生产每盒产品的可变成本等于盒子的体积。生产任何特定盒子的固定成本为 1000 美元。如果公司要求一个盒子装的物体的体积必须小于盒子的体积，那么公司应该如何制定包装盒的生产计划，使得以最小的成本满足所有的需求？

包装盒类型	1	2	3	4	5	6	7
容量（V）	33	30	26	24	19	18	17
需求量（d）	400	300	500	700	200	400	200

我们引入下面的变量。

y_i：0-1 变量；如果生产第 i 种盒子，$y_i=1$，否则，$y_i=0$。

x_i：整数变量；第 i 种盒子的生产数量。

模型如下：

$$\min \quad 1000\sum_{i=1}^{7} y_i + \sum_{i=1}^{7} V_i x_i \tag{5.11}$$

$$\text{s.t.} \quad \sum_{i=1}^{j} V_i x_i \geqslant \sum_{i=1}^{j} d_i, \quad \forall j \in \{1,2,\cdots,7\} \tag{5.12}$$

$$x_i - M y_i \leqslant 0, \quad \forall i \in \{1,2,\cdots,7\} \tag{5.13}$$

$$x_i \geqslant 0, x_i \in \mathbb{Z}, \quad y_i \in \{0,1\} \tag{5.14}$$

其中，M 是一个很大的正数。

我们用 Java 调用 CPLEX 来求解上面的问题。代码如下：

CPLEX Java Interface

```
package Example2;

import ilog.concert.*;
import ilog.cplex.*;

import java.io.*;
import java.util.*;

public class Main {

        public static void main(String[] args) throws IOException,
      IloException {

              double[] cost = {33, 30, 26, 24, 19, 18, 17};
              double[] demand = {400, 300, 500, 700, 200, 400, 200};
              IloCplex Cplex = new IloCplex();  //这里需要抛异常

              IloNumVar[] X = new IloNumVar[cost.length];
              IloNumVar[] Y = new IloNumVar[cost.length];
              for(int i = 0; i < cost.length; i++){
                      Y[i] = Cplex.boolVar("Y" + i);
                      X[i] = Cplex.intVar(0, 1000, "X" + i);
              }

              IloNumExpr obj = Cplex.numExpr();
              for (int i = 0; i < cost.length; i++) {
                      obj = Cplex.sum(obj, Cplex.prod(1000, Y[i]));
                      obj = Cplex.sum(obj, Cplex.prod(cost[i], X[i]));
              }

              Cplex.addMinimize(obj);

              IloNumExpr expr = Cplex.numExpr();
              //Constraint 1:
              double totaldemand = 0;
              for(int j = 0; j < cost.length; j++){
                      expr = Cplex.numExpr();
                      totaldemand = totaldemand + demand[j];
                      for(int i = 0; i <= j; i++) {
                          expr = Cplex.sum (expr, Cplex. prod (cost[i],
        X[i]));
```

```
                }
                Cplex.addGe(expr, totaldemand);
        }


        //Constraint 2:
        expr = Cplex.numExpr();
        for(int i = 0; i < cost.length; i++){
                expr = Cplex.sum(expr, X[i]);
                expr = Cplex.sum(expr, Cplex.prod(-100000, Y[i]));
                Cplex.addLe(expr, 0);
                expr = Cplex.numExpr();
        }


        //Solve the model
        Cplex.exportModel("SPP.lp");
        Cplex.solve();


        //------------------Solution-------------------------
        System.out.println("\n\n----------Solution---------\n");
        System.out.println("Objective\t\t:" + Cplex.getObjValue());
        for (int i = 0; i < cost.length; i++) {
                if(Cplex.getValue(X[i]) > 0) {
                        System.out.println("x[" + i + "] = " + Cplex
.getValue(X[i]) );
                }
        }

        for (int i = 0; i < cost.length; i++) {
                if(Cplex.getValue(Y[i]) > 0) {
                        System.out.println("y[" + i + "] = " + Cplex
.getValue(Y[i]));
                }
        }
    }
}
```

得到的最优解如下：

Results

```
----------The solution---------

Objective                :3706.0
x[0] = 82.0
y[0] = 1.0
```

也就是说，该公司需要生产 82 个体积为 33 立方英寸的盒子，总成本为 3706 美元。

第6章 Gurobi的Python API详解及简单案例

本章主要介绍 Gurobi 的 Python 接口的使用方法以及 Gurobi 的算法框架。其中大量内容参考自 Gurobi 官方用户手册和官方教程资料（参见网址https://www.gurobi.com），为方便阅读起见，我们对具体参考位置不做特殊标注。

6.1 Python 调用 Gurobi 环境配置

6.1.1 完整步骤

首先，需要安装 Gurobi 和 Python。但是需要注意以下几点。

（1）Gurobi 的版本和 Python 的版本须一致，即都是 64 bit 或者都是 32 bit，否则不能成功。

（2）如果用 PyDev，则需要 Eclipse、Gurobi 和 Python 三者版本是一样的，都是 64 bit 或者 32 bit。

以下是详细步骤。

1. Step 1

找到 gurobi 安装目录 C:\Develop\Gurobi81\win64\python27\lib 中的文件夹 gurobipy （注意是文件夹），如图 6.1 所示。

2. Step 2

将 gurobipy 文件夹复制到 Python 的安装目录下，即 C:\Develop\Python27_64\Lib，如图 6.2 所示。（注意：这是笔者自己的路径，如果是读者的，读者需要查看自己的安装目录。）

6.1.2 相关问题

如果出现问题，可能是如下原因。

一是 Gurobi 的 License 期限到了，需要重新申请。

二是环境变量或者版本的问题。

1. License 到期

若 License 到期，则需要重新申请 License，此时要填写一个申请表格发送给 help@gurobi.cn。

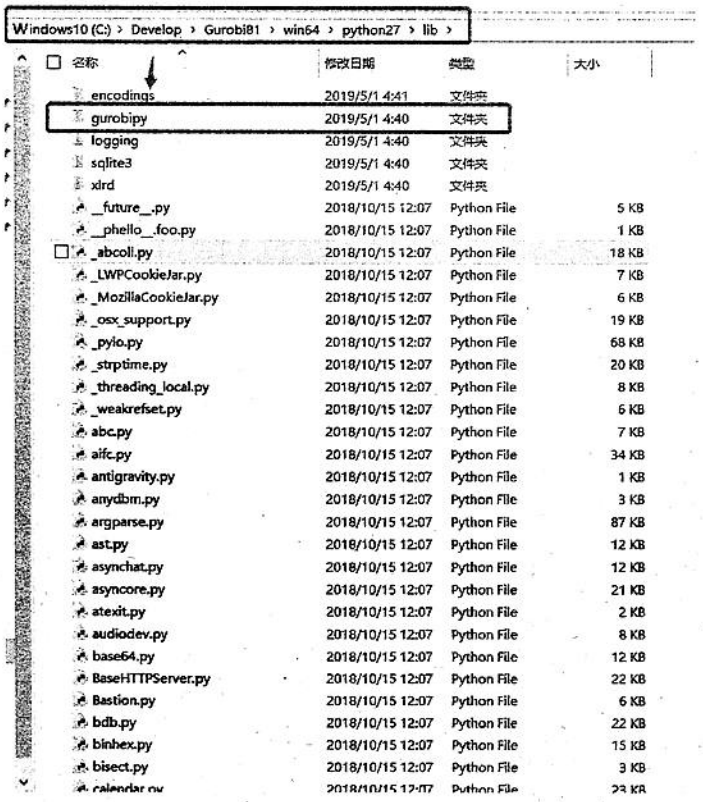


图 6.1 gurobipy 文件夹的完整路径

申请要求如下：高校教师或者在校学生可以申请 Gurobi 一年的免费使用权，到期后可以继续申请。但是仅对教师和学生开放，并且不能用于商业用途。

申请步骤如下。

（1）下载申请表格。

（2）请用英文填写申请表，签字后以 PDF 格式保存。命名为 XXX Form.pdf，其中 XXX 是申请人姓名的汉语拼音。（文件大小不要超过 1 MB）

（3）以 PDF 格式保存学生证/教师证扫描件。命名为 XXX ID.pdf, 其中 XXX 是申请人姓名的汉语拼音。（文件大小不要超过 1 MB）

（4）需要由本人提出申请。如果联系邮箱和申请人姓名不一致，或者无法证明提供的资料和申请人是同一个人，将要求申请表格由系或者学院盖章证明。

（5）将申请表和身份扫描件通过邮件发到 help@gurobi.cn。

（6）处理时间为收到完整资料后 2 个工作日（一般工作日上午申请，下午就可以得到 License）。

注意：除了图 6.3 中方框内的签名，其他信息首先在电子版内填写完毕，然后打印出来。接下来在纸质版中对应红框位置处手写姓名的拼音，然后扫描（推荐使用扫描全能王

App）得到 PDF 版。最后将上述表格的 PDF 文件和学生证/教师证的 PDF 文件一起发送至 help@gurobi.cn 即可。

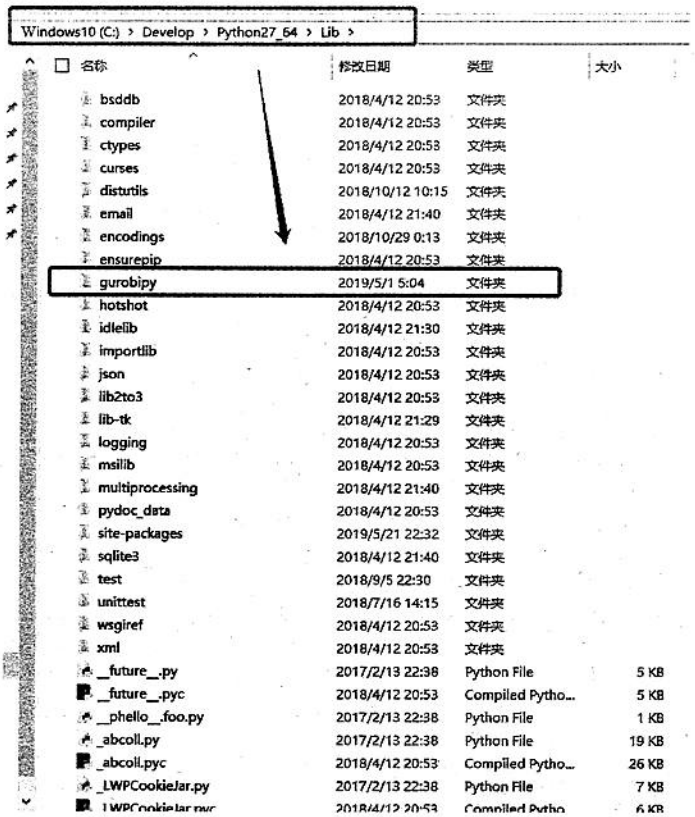


图 6.2 将 gurobipy 文件夹复制到 Python 的安装目录下

表格的图片如图 6.3 所示。

2. 环境变量或者版本的问题

如果是环境变量或者版本的问题，可能会报如图 6.4 所示的错误。如果报以上错误，可以通过下面的方法解决。

（1）在高级系统设置中找到环境变量 Path，如图 6.5 所示。

（2）在 Path 中添加 Gurobi 和 Python 的安装路径，如图 6.6 所示。

但是上述问题一般不会存在。

2019 年 4 月 30 日，笔者曾遇见这样的问题，笔者的解决方法如下。

Step1：卸载原来的 Gurobi 8.1。

Step2：重新安装 Gurobi 8.1。

Personal Information

Full name: [redacted]
Email address: [redacted]
Phone number: [redacted]

Address: Graduate School at ShenZhen, Tsinghua University,
Tsinghua Campus, The University Town, Shenzhen, P. R. China
City: Shenzhen
State/Province: Guangdong
Zip/Postal Code: 518055
Country: China

Academic Information

Academic Institution: Graduate School at ShenZhen, Tsinghua University
Website address: www.sz.tsinghua.edu.cn

Program or Department Information

Classification: (●) Student ( ) Faculty ( ) Research Staff ( ) Administrative Staff

Department name: Industrial Engineering
Department phone: 86-0755-2603
If student: graduation date: [redacted]

Please note that the email address above must match the address used to create your account on www.gurobi.com.

By signing below you attest that you are eligible to receive a free academic license, will only use it for research or academic purposes, and understand that it will expire twelve months after the date the license is generated.

[redacted] Printed Name　　[redacted] Signature

图 6.3 申请学术 License 的表格

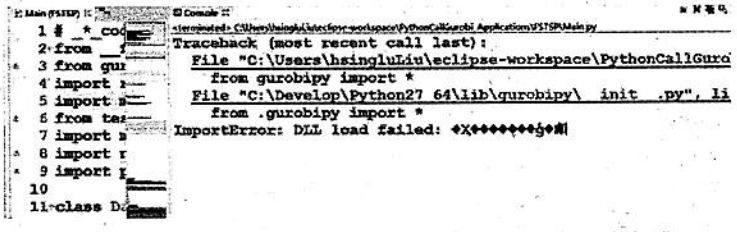


图 6.4 Eclipse 中用 PyDev 实现 Python 调用 Gurobi 的报错

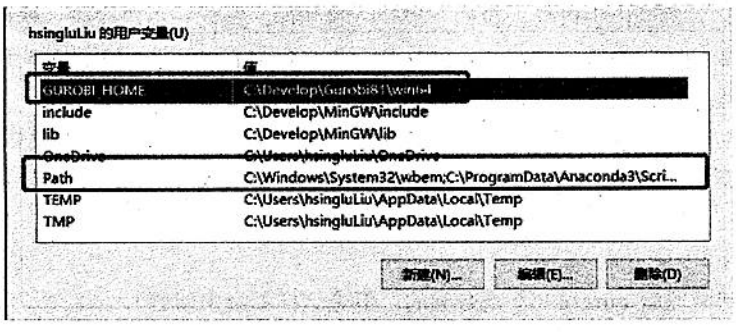


图 6.5 在高级系统设置中找到环境变量 Path

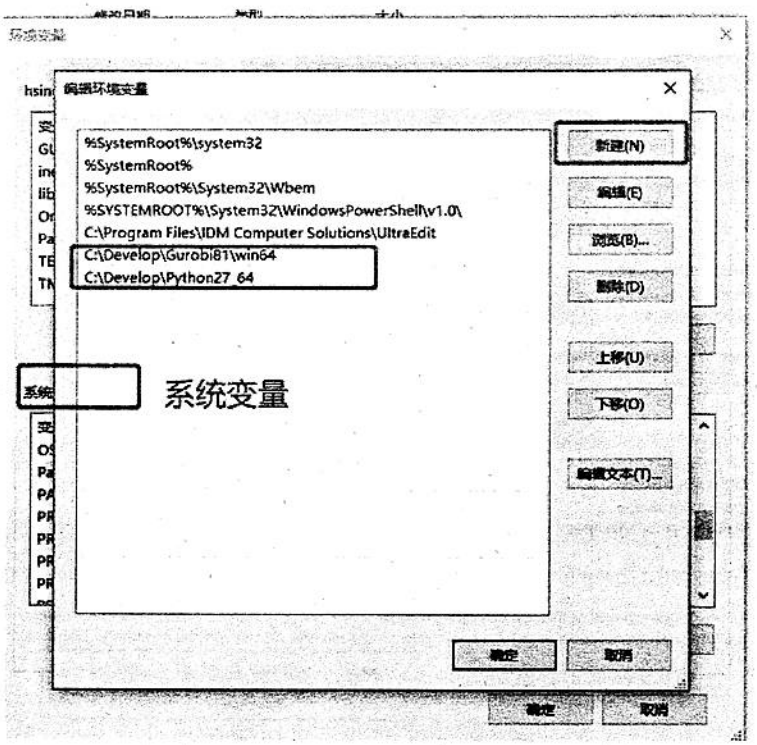


图 6.6 在 Path 中添加 Gurobi 和 Python 的安装路径

Step3：将 Gurobi 的安装目录，即 C:\Develop\Gurobi\win64\python27\lib 中的文件夹 gurobipy（注意是文件夹），复制到 Python 的安装路径的 Lib 文件夹中，即 C:\Develop\Python27_64\Lib。

Step4：重启 Eclipse，就可以了。

这种情况下，一般是 Gurobi 的 License 出了问题，所以需要重新安装一下，更新 License。

3. License 的获取以及更新 Gurobi 的版本

填完表格并发送出去以后，我们会得到一封邮件，运行一小段代码，然后就会在特定的目录下生成一个名为 gurobi.lic 的文件，将其复制到 Gurobi 的安装目录下即可。

具体流程分别如图 6.7～ 图 6.10 所示。

如果需要更新版本，可以选择先把原来版本卸载，重新安装，然后再重新运行 License 口令，生成新的 gurobi.lic 文件，然后在命令行窗口输入 gurobi 查看版本信息，如图 6.11 所示。

步骤如下。

（1）先卸载原版 gurobi（不是必须的，但是这样不会出问题）。

（2）重新运行 License 口令。

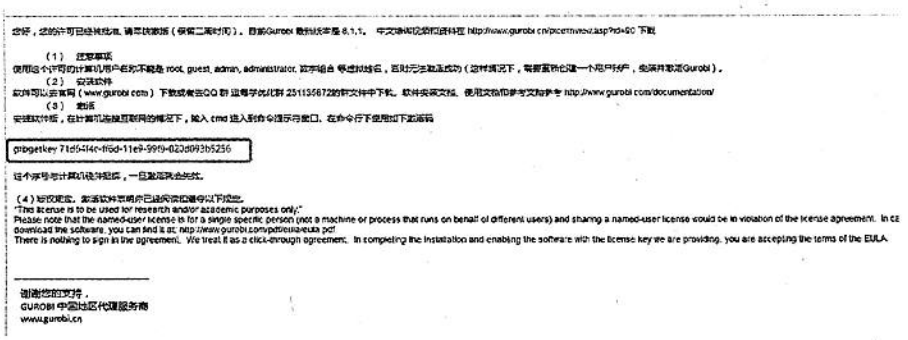


图 6.7 Gurobi 的 License 的安装：邮件

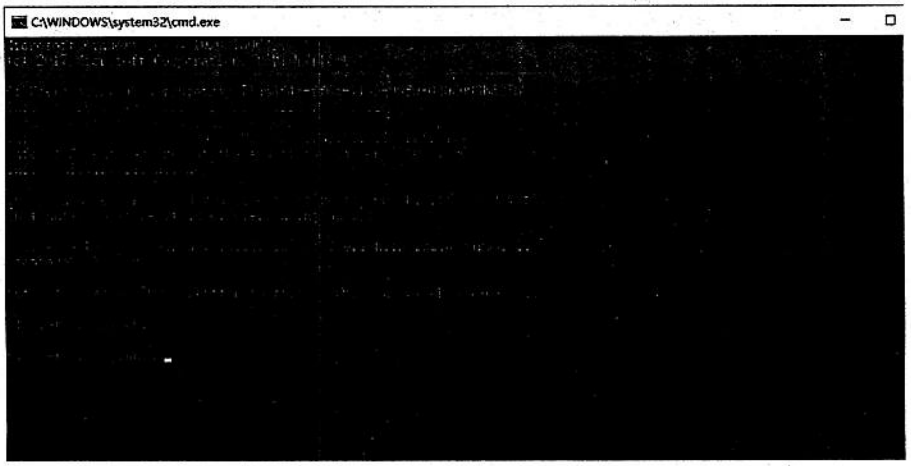


图 6.8 Gurobi 的 License 的安装：命令行窗口运行口令

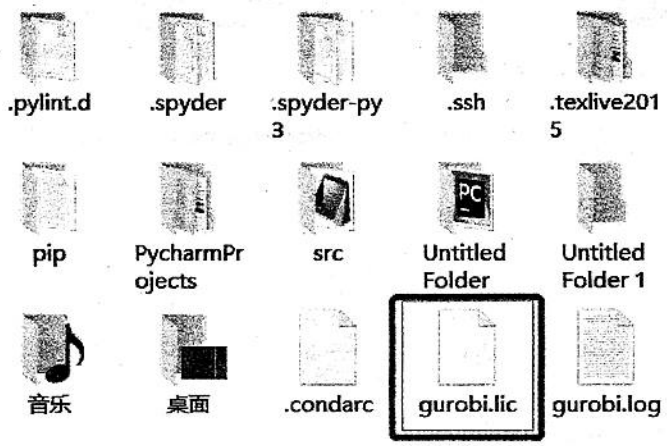


图 6.9 Gurobi 的 License 的安装：生成 gurobi.lic 文件



图 6.10 Gurobi 的 License 的安装：将 gurobi.lic 文件复制至 Gurobi 的安装目录

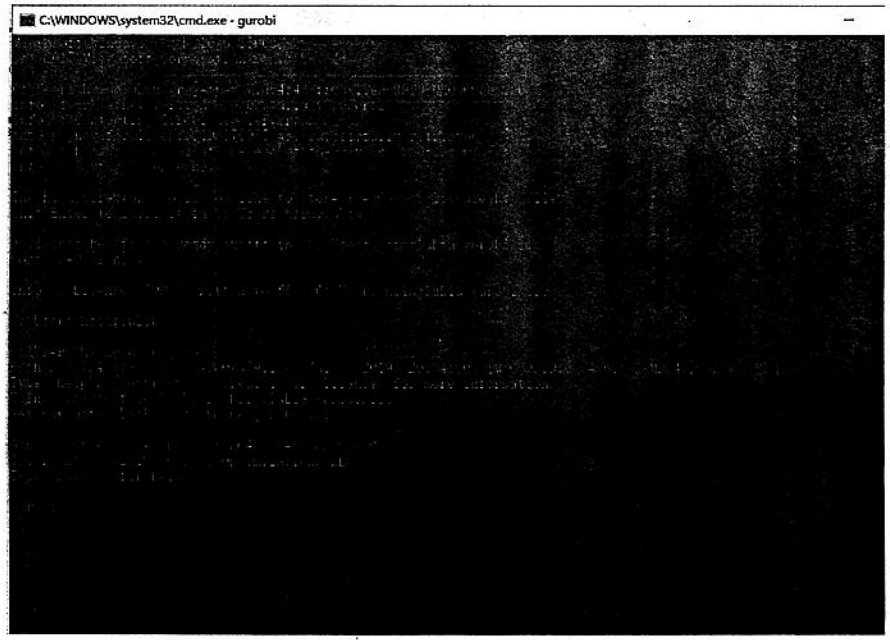


图 6.11 Gurobi 的 License 的安装：更新 Gurobi 版本成功后的效果查看

6.2 Gurobi 算法框架介绍

6.2.1 Gurobi MIP Algorithms

我们首先看一下 Gurobi 的 MIP 算法框架，如图 6.12 所示。

Gurobi 与 CPLEX 一样，算法的主体框架都是分支切割，如图 6.12所示。在求解模型之前，首先根据一些预处理的方法，将模型中的冗余变量和冗余约束删除，以精简模型，这个过程叫作 Presolve（预求解），具体的理论见 Tobias Achterberg，Robert E. Bixby，Zonghao Gu Edward Rothberg，Dieter Weninger 在 2020 年发表在 *INFORMS Journal on Computing* 上题为 *Presolve Reductions in Mixed Integer Programming* 的文章（Achterberg

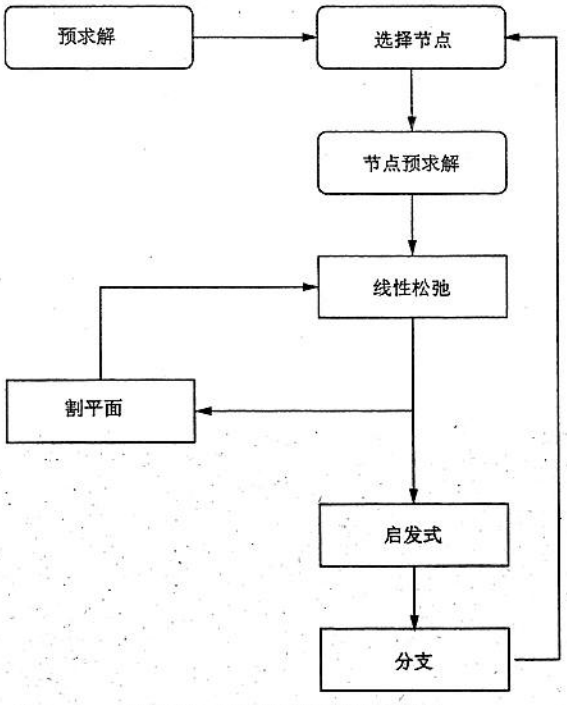


图 6.12 Gurobi MIP 算法框架

et al., 2020)。特别地，Tobias Achterberg，Robert E. Bixby，Zonghao Gu Edward Rothberg 均是 Gurobi 团队成员，同时，后 3 位还是 Gurobi 的联合创始人。此外，上述 4 位曾经都是 CPLEX 团队成员，其中，Tobias Achterberg 还是目前最好的开源求解器——SCIP 的创始人。

回到算法框架部分。Presolve 之后，就是整个 Branch and Cut 的主体部分。该部分主要分为基于线性规划的分支定界（LP based Branch and Bound）、割平面（Cutting Plane）和启发式算法（Heuristic）。基于线性规划的分支定界是只基于 LP 模型自身的变量去做分支，不依赖具体问题特性。Gurobi 是面向通用模型设计的求解器，不能实现自动按照问题特性构造特定的分支规则（例如 VRPTW 中根据资源窗口进行分支的操作，Gurobi 等求解器是不能自动做到的）。割平面是为了在每个节点加快逼近该节点的凸包，从而加快求解速度。启发式算法是在求解完节点的线性松弛（LP Relaxation）之后，利用启发式算法快速找到较好的可行解，以提升 Bound，进而提高剪枝（Prune）效率。

下面是分支切割算法框架的完整过程。首先是选择当前需要处理的节点（Node Selection），同样地，在被选中的节点处执行预求解操作（Node Presolve）。接下来就是分支切割算法的主体部分。首先是将节点的模型做线性松弛，求解该线性松弛并得到相应的解。此时，Gurobi 会调用割平面算法生成一系列的 Cuts，并将这些 Cuts 加回到当前节点的线性松弛中，再次求解加入 Cuts 的线性松弛，得到相应的解，如果该解为整数，则执行剪枝操

作；如果该解为小数则执行启发式算法，利用 Gurobi 的启发式算法快速找到一些可行整数解，辅助更新 Bound。之后就执行分支操作，生成分支子节点。循环该过程，直到算法终止。

接下来，我们对图 6.12中的各个部分给出更为详细的介绍。

6.2.2 Presolving

Gurobi 中的预求解（Presolve）分为两个阶段：根节点预求解（Root Presolve）和节点预求解（Node Presolve）。根节点预求解是指在第一次线性规划松弛的求解之前应用的预求解操作，而节点预求解是指在分支定界搜索树的节点上应用的预求解操作。如图 6.13 所示，每次预求解后模型的行列将会缩减。

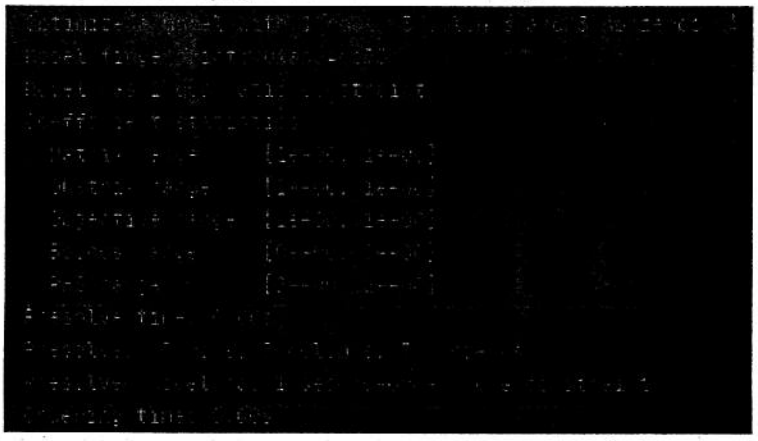


图 6.13 Gurobi 中的预优化操作

预求解操作主要包括以下几项。

（1）单个约束的约简。其中包括清理模型，删除冗余约束（Model Cleanup and Removal of Redundant Constraints）；加强 Bound（Bound Strengthening）；加强系数（Coefficient Strengthening）；不等式约束的 Chvatal-Gomory 加强（Chvatal-Gomory Strengthening of Inequalities）等。

（2）单个变量的约简。其中包括固定变量的去除（Removal of Fixed Variables）、整数变量的圆整 Bounds（Rounding Bounds of Integer Variables）、加强半连续和半整数 Bounds（Strengthen Semi-continuous and Semi-integer Bounds）等。

（3）同时考虑多个约束的约简。其中包括冗余检测（Redundancy Detection）、平行和近平行行（Parallel and Nearly Parallel Rows）等。

（4）同时考虑多个变量的化简。其中包括平行列（Parallel Columns）、主导列（Dominated Columns）等。

（5）考虑整个问题的化简。其中包括对称变量的聚合对（Aggregate Pairs of Symmetric

Variables）和 Probing 等。

6.2.3 Node Selection

Gurobi 求解器中节点选择的步骤就是确定分支树中节点遍历顺序的过程，如图 6.14 所示[①]。

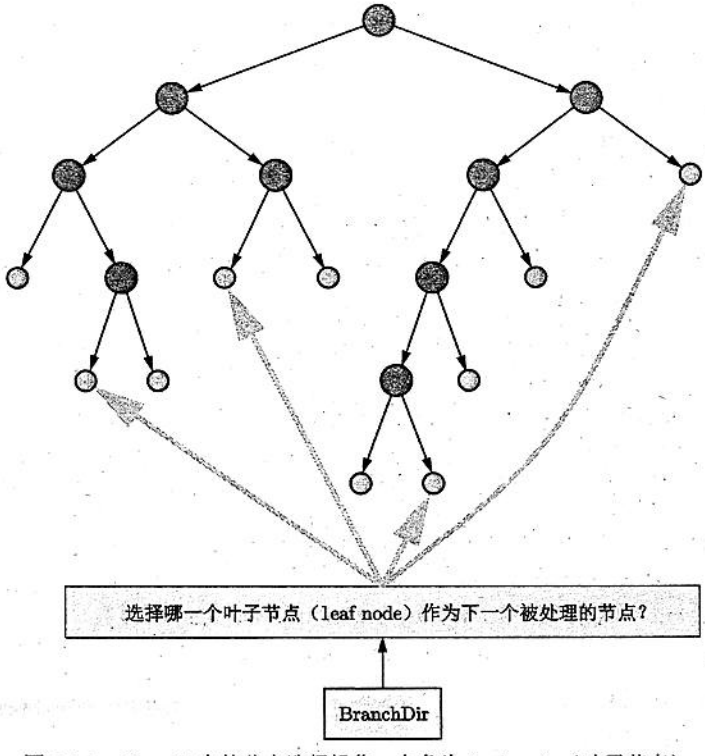


图 6.14 Gurobi 中的节点选择操作，灰色为 leaf node（叶子节点）

图中 BranchDir 是分支偏好参数（Preferred branch direction）。其实就是决定下一个被处理的节点（node）在分支切割树（Branch and cut tree）当中的位置。该值默认为 0，其他可选值为 −1 和 1。当该值为默认值时，表示 Gurobi 自动选择。当该值为 −1 时，表示选择分支切割树最下层的节点作为下一个要处理的节点，也就是所谓的深度优先（DFS）；当该值为 1 时，表示选择分支切割树最上层的节点作为下一个要处理的节点，也就是所谓的广度优先（BFS）。

分支切割树中被存储下来的，一定都是目前为止的树中的叶子节点（leaf node）。选好了下一个要被处理的节点之后，就可以选择该节点中的解中取值为小数的变量中的某一个进行分支，继续迭代算法。或者是选择某几个，根据其他分支策略进行分支。

① 本图参考自 Gurobi 官方教程，见网址http://www.gurobi.cn/download/gu20180713.pdf。

6.2.4 Cutting Planes

Gurobi 求解器公布的割平面一共有下面 19 种。在求解整数规划或者混合整数规划等模型时，只要有整数变量，Gurobi 就会自动调用下面的割平面算法，并且在求解结果输出的时候输出每一种割平面算法使用的次数。

下面是 19 种 Gurobi 使用的割平面算法。

(1) BQP cut (Boolean Quadric Polytope cut)。

(2) Clique cut。

(3) Cover cut。

(4) Flow cover cut。

(5) Flow path cut。

(6) Gomory cut (Chvátal-Gomory cut)。

(7) GUB cover cut (Generalized Upper Bound cover cuts)。

(8) Implied bound cut。

(9) Infeasibility proof cut。

(10) MIP separation cut。

(11) MIR cut (Mixed Integer Rounding cut)。

(12) Mod-k cut。

(13) Network cut。

(14) Projected implied bound cut。

(15) Relax-and-lift cut。

(16) RLT cut (Reformulation Linearization Technique cut)。

(17) Strong-CG cut (Strong Chvátal-Gomory cut)。

(18) Sub-MIP cut。

(19) Zero-half cut。

具体每种算法的原理在这里不做展开。这里仅给出一个整数规划的求解结果输出示例，如下所示：

Gurobi MIP Output

1	H	0	0				28.1409947	28.64016	1.77%	-	1s
2		0	0	28.61517	0	66	28.14099	28.61517	1.69%	-	1s
3		0	0	28.61517	0	95	28.14099	28.61517	1.69%	-	1s
4		0	0	28.61517	0	90	28.14099	28.61517	1.69%	-	1s
5		0	0	28.61517	0	98	28.14099	28.61517	1.69%	-	1s
6		0	0	28.61517	0	99	28.14099	28.61517	1.69%	-	1s
7		0	0	28.60396	0	102	28.14099	28.60396	1.65%	-	2s
8		0	0	28.60396	0	101	28.14099	28.60396	1.65%	-	2s
9		0	2	28.57160	0	101	28.14099	28.57160	1.53%	-	2s
10											

```
Cutting planes:
  Gomory: 2
  Cover: 6
  Implied bound: 34
  Clique: 1
  MIR: 74
  Flow cover: 48
  GUB cover: 6
  Zero half: 1
  RLT: 9
  Relax-and-lift: 15

Explored 121 nodes (12969 simplex iterations) in 2.97 seconds
Thread count was 8 (of 8 available processors)
```

可以从上述输出信息中看出哪些割平面算法被调用了，以及调用了多少次。

6.2.5 Heuristics

Gurobi 集成了 30 多种启发式算法。有些启发式算法是基于线性规划设计的，有些是独立于线性规划设计的，还有一些其他的启发式算法。这里我们做简要介绍。

1. Not based on LP

独立于线性规划的启发式算法有贪婪算法（Greedy Algorithm）、动态规划（Dynamic Programming）、最短路径（Shortest Path）、最小生成树（Minimum Spanning Tree）和一些 Gurobi 的 blind heuristics（盲目启发式），blind 的意思是不启用 LP relaxation 的解。这些不依赖于线性规划的启发式算法的目的是快速地找到一个可行解。

2. Based on LP

另外一些基于线性规划的启发式算法有：圆整（Rounding，一般来讲不是很有效，很难对方程进行圆整）；基于 LP 的贪婪算法等。

3. Reformulation

模型重构（解决可行性问题）：删除目标并更改约束条件，例如：

$$\boldsymbol{Ax} - \boldsymbol{r} \leqslant \boldsymbol{b} \Rightarrow \boldsymbol{Ax} + \boldsymbol{s} - \boldsymbol{t} = \boldsymbol{b}$$

设置目标函数为

$$\min \quad \boldsymbol{s} + \boldsymbol{t}$$

4. RINS

松弛诱导邻域搜索（Relaxation Induced Neighborhood Search，RINS）是一种改进型启发式算法，是 Gurobi 中最有效的启发式算法。

给定现任整数解和节点松弛的当前分数解，如果变量的整数解值和松弛解值一致，则固定变量，将部分固定的模型作为子 MIP 去解。

5. 子 MIP 和递归方法

子 MIP 和递归方法通常非常有效，可迅速找到可行解。

6. 泵式缩减启发式

详见 gurobi 用户手册。

7. 泵式缩减

详见 gurobi 用户手册。

6.2.6 设置启发式的参数

根据经验设置启发式的频率。

有三种参数类型：（1）MIP parameters；（2）Main heuristic parameters；（3）Some heuristic parameters。

有三种 MIPFocus：（1）good feasible solution；（2）optimal solution；（3）bound。

上述 3 个 MIPFocus 值分别对应 MIP 求解的 3 个侧重点：侧重快速找到可行解；侧重证明最优；侧重界的提升（发现界提升缓慢）。可以通过 `Model.Params.MIPFocus` 设定 MIP 求解的侧重点。

默认：5% 的算法使用启发式。

6.2.7 Branching

Gurobi 求解器中的分支操作就是根据当前松弛问题的小数解选择分支变量。分支变量的选择是限制搜索树大小的关键。图 6.15 中的 IntInf 展示了 Gurobi 在当前节点的线性松弛模型的解中，整数决策变量取了小数值的变量的个数。

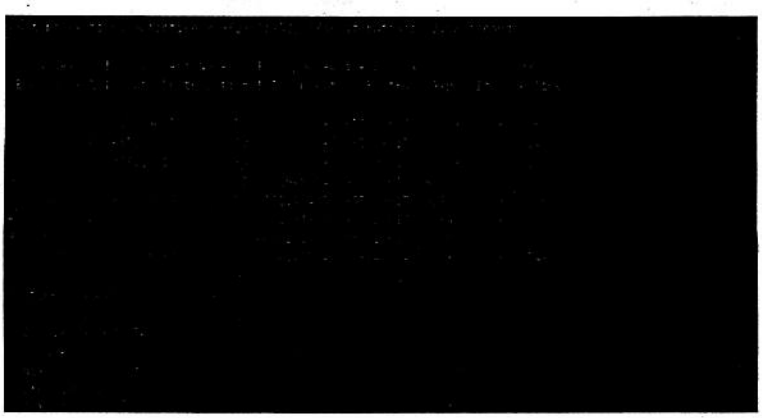


图 6.15 Gurobi 分支输出信息

分支操作主要如下。

1. 变量分支

(1) Max fractional value：取分数部分最大的变量作为分支变量。该方法效果不佳，甚至比随机选取还差。

(2) Shadow costs (similar to pseudo costs)：影子成本。将变量向上分支和向下分支，并重新求解分支后的线性松弛，评估分支对目标值的单位影响，计算出影子成本。最后选择影子成本最大的变量进行分支。

(3) Strong branching：强分支。首先求解分支变量对应的子节点的线性松弛，然后选择引起目标函数变化最大的变量作为分支变量。

(4) "Modern" pseudo costs：对于分支变量对目标的影响做持续估计。使用强分支做初始化来改进，并多次运用强分支做进一步改进（可靠性分支）。

2. SOS 分支

Pseudo-cost branching for SOS sets：通过固定为零并求解 LP 来计算可变伪成本；为松弛解为 x^* 的不可行 SOS 集合 $x_1, x_2, \cdots, x_k$ 找到一个分割；分别计算左右两个集合的伪成本，sumpcost$[i] \times x[i]$，然后相加；找到相加之和最大的集合。

6.3 Gurobi 能够求解的模型类别

6.3.1 线性规划

线性规划（Linear Programming，LP）是 Gurobi 能求解的最基本的数学规划。其一般形式如下：

$$\begin{aligned} \min \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b} \\ & \boldsymbol{x} \in \mathbb{R}^n \end{aligned} \tag{6.1}$$

下面是 Gurobi 求解线性规划的一个简单代码案例：

LinearProblem.lp

```
\ Model LinearProblem
\ LP format - for model browsing. Use MPS format to capture full model
    detail.
Maximize
3 x + 5 y + 4 z
Subject To
c0: 2 x + 3 y <= 15
c1: 2 y + 4 z <= 8
c2: 3 x + 2 y + 5 z >= 2
Bounds
End
```

Gurobi Python Interface

```
from gurobipy import *

try:

    # Create a new model
    m = Model("LinearProblem")

    # Create variables
    x = m.addVar(vtype=GRB.CONTINUOUS, name="x")
    y = m.addVar(vtype=GRB.CONTINUOUS, name="y")
    z = m.addVar(vtype=GRB.CONTINUOUS, name="z")

    # Set objective
    m.setObjective(3 * x + 5 * y + 4 * z, GRB.MAXIMIZE)

    # Add constraint
    m.addConstr(2 * x + 3 * y <= 15, "c0")
    m.addConstr(2 * y + 4 * z <= 8, "c1")
    m.addConstr(3 * x + 2 * y + 5 * z >= 2, "c2")

    # Write model to file
    m.write("LinearProblem.lp")

    # Solve the model
    m.optimize()

    print('Optimal solution', end = "    ")
    for i in m.getVars():
        print('%s = %g' % (i.varName, i.x), end = " ")

except GurobiError as e:
    print('Error code ' + str(e.errno) + ": " + str(e))

except AttributeError:
    print('Encountered an attribute error')
```

求解结果如下：

Gurobi Python Interface

```
Optimal objective  3.050000000e+01
Optimal solution   x = 7.5 y = 0 z = 2
```

6.3.2 混合整数规划

Gurobi 也可以求解混合整数规划(Mixted Integer Programming, MIP),其一般模型为

$$\begin{aligned} \min \quad & m^{\mathrm{T}}x + n^{\mathrm{T}}y \\ \text{s.t.} \quad & Ax \leqslant b \\ & Cy \leqslant d \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^n \end{aligned} \tag{6.2}$$

下面是一个 Gurobi 求解混合整数规划的简单代码案例:

IntegerProblem.lp

```
\ Model IntegerProblem
\ LP format - for model browsing. Use MPS format to capture full model
    detail.
Maximize
3 x + 5 y + 4 z
Subject To
c0: 2 x + 3 y <= 15
c1: 2 y + 4 z <= 8
c2: 3 x + 2 y + 5 z >= 2
Bounds
Generals
x y z
End
```

Gurobi Python Interface

```
from gurobipy import *

try:

    # Create a new model
    m = Model("IntegerProblem")

    # Create variables
    x = m.addVar(vtype=GRB.INTEGER, name="x")
    y = m.addVar(vtype=GRB.INTEGER, name="y")
    z = m.addVar(vtype=GRB.INTEGER, name="z")

    # Set objective
    m.setObjective(3 * x + 5 * y + 4 * z, GRB.MAXIMIZE)
```

```
15
16      # Add constraints
17      m.addConstr(2 * x + 3 * y <= 15, "c0")
18      m.addConstr(2 * y + 4 * z <= 8, "c1")
19      m.addConstr(3 * x + 2 * y + 5 * z >= 2, "c2")
20
21      # Write model to file
22      m.write("IntegerProblem.lp")
23
24      # Solve the model
25      m.optimize()
26
27      print('Optimal solution', end = "    ")
28      for i in m.getVars():
29          print('%s = %g' % (i.varName, i.x), end = " ")
30
31  except GurobiError as e:
32      print('Error code ' + str(e.errno) + ": " + str(e))
33
34  except AttributeError:
35      print('Encountered an attribute error')
```

求解结果如下：

Gurobi Python Interface

```
1  Best objective 2.900000000000e+01, best bound 2.900000000000e+01, gap
       0.0000%
2  Optimal solution    x = 7 y = -0 z = 2
```

6.3.3 二次规划

Gurobi 也可以用于求解二次规划（Quadratic Programming，QP），其一般形式为

$$\begin{aligned} \min\ & \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{x}+\boldsymbol{q}^{\mathrm{T}}\boldsymbol{x}+r \\ \text{s.t.}\ & \boldsymbol{G}\boldsymbol{x}\leqslant\boldsymbol{h} \\ & \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \end{aligned} \tag{6.3}$$

下面是一个 Gurobi 求解二次规划的简单代码案例：

QuadraticProblem.lp

```
1  \ Model QuadraticProblem
2  \ LP format - for model browsing. Use MPS format to capture full model
       detail.
```

```
Minimize
- 2 x - 6 y + [ x ^2 - 2 x * y + 2 y ^2 ] / 2
Subject To
c0: - x + 2 y <= 2
c1: x + y <= 2
c2: 2 x + y <= 3
Bounds
End
```

Gurobi Python Interface

```
from gurobipy import *

try:

    # Create a new model
    m = Model("QuadraticProblem")

    # Create variables
    x = m.addVar(lb = 0, vtype=GRB.CONTINUOUS, name="x")
    y = m.addVar(lb = 0, vtype=GRB.CONTINUOUS, name="y")

    # Set objective
    m.setObjective(1/2 * x * x + y * y - x * y - 2 * x - 6 * y, GRB.
      MINIMIZE)

    # Add constraints
    m.addConstr(-x + 2 * y <= 2, "c0")
    m.addConstr(x + y <= 2, "c1")
    m.addConstr(2 * x + y <= 3, "c2")

    # Write model to file
    m.write("QuadraticProblem.lp")

    # Solve the model
    m.optimize()

    print('Optimal solution', end = "    ")
    for i in m.getVars():
        print('%s = %g' % (i.varName, i.x), end = " ")

except GurobiError as e:
    print('Error code ' + str(e.errno) + ": " + str(e))
```

```
32
33 except AttributeError:
34     print('Encountered an attribute error')
```

或者说可以用 QuadExpr 来创建目标函数（当然也可以创建约束）。即目标函数部分的代码改成如下形式：

Gurobi Python Interface

```
1  # creat objective
2  obj = QuadExpr(0)
3  obj.addTerms(-2, x)
4  obj.addTerms(-6, y)
5  obj.addTerms(0.5, x, x)
6  obj.addTerms(1, y, y)
7  obj.addTerms(-1, x, y)
8
9  # Set objective
10 m.setObjective(obj, GRB.MINIMIZE)
11
12 # 等价于
13 # m.setObjective(1/2 * x * x + y * y - x * y - 2 * x - 6 * y, GRB.MINIMIZE)
```

如果说规模比较大的问题，就可以使用循环来拼接表达式，完成复杂表达式或者约束的创建。

上述问题的求解结果如下：

Gurobi Python Interface

```
1 Optimal objective -8.22222222e+00
2 Optimal solution   x = 0.666667 y = 1.33333
```

6.3.4 二次约束二次规划

二次约束二次规划（Quadratically Constrained Quadratic Programming，QCQP）也可以用 Gurobi 进行求解。其一般形式为

$$\begin{aligned}\min\ & \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}_0\boldsymbol{x}+\boldsymbol{q}_0^{\mathrm{T}}\boldsymbol{x}+r_0\\ \text{s.t.}\ & \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}_i\boldsymbol{x}+\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{x}+r_i\leqslant 0,\forall i=1,2,\cdots,m\\ & \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}\end{aligned}\tag{6.4}$$

下面是一个 Gurobi 求解二次约束二次规划的简单代码案例：

QCQP.lp

```
\ Model QCQP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
- 2 x - 6 y + [ x ^2 - 2 x * y + 2 y ^2 ] / 2
Subject To
c0: - x + 2 y <= 2
c2: 2 x + y <= 3
c1: 0.5 y + [ x ^2 ] <= 2
Bounds
End
```

Gurobi Python Interface

```
from gurobipy import *

try:

    # Create a new model
    m = Model("QCQP")

    # Create variables
    x = m.addVar(lb = 0, vtype=GRB.CONTINUOUS, name="x")
    y = m.addVar(lb = 0, vtype=GRB.CONTINUOUS, name="y")

    # Set objective
    m.setObjective(1/2*x*x+y*y-x*y-2*x-6*y, GRB.MINIMIZE)

    # Add constraints
    m.addConstr(-x + 2 * y <= 2, "c0")
    m.addConstr(x * x + 1/2 * y <= 2, "c1")
    m.addConstr(2 * x + y <= 3, "c2")

    # Write model to file
    m.write("QCQP.lp")

    # Solve the model
    m.optimize()

    print('Optimal solution', end = "    ")
    for i in m.getVars():
        print('%s = %g' % (i.varName, i.x), end = " ")
```

```
29
30 except GurobiError as e:
31     print('Error code ' + str(e.errno) + ": " + str(e))
32
33 except AttributeError:
34     print('Encountered an attribute error')
```

当然，仍然可以用前面介绍过的 QuadExpr 来创建目标函数、约束等。

上述问题的求解结果如下：

Gurobi Python Interface

```
1 Optimal objective -8.83999672e+00
2 Optimal solution   x = 0.8 y = 1.4
```

6.3.5 二阶锥规划

二阶锥规划（Second-Order Cone Programming，SOCP）也可以用 Gurobi 进行求解，其一般形式为

$$\begin{aligned} \min\ & \boldsymbol{f}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.}\ & \|\boldsymbol{A}_i\boldsymbol{x}+\boldsymbol{b}_i\|_2 \leqslant \boldsymbol{c}_i^{\mathrm{T}}\boldsymbol{x}+d_i, i=1,2,\cdots,m \\ & \boldsymbol{F}\boldsymbol{x}=\boldsymbol{g} \end{aligned} \tag{6.5}$$

下面是一个 Gurobi 求解二阶锥规划的简单代码案例：

SOCP.lp

```
1 \ Model SOCP
2 \ LP format - for model browsing. Use MPS format to capture full model
      detail.
3 Maximize
4 x
5 Subject To
6 c0: x + y + z = 2
7 c1: [ x ^2 + y ^2 - z ^2 ] <= 0
8 Bounds
9 End
```

Gurobi Python Interface

```
1 from gurobipy import *
2
3 try:
```

```
4
5       # Create a new model
6       m = Model("SOCP")
7
8       # Create variables
9       x = m.addVar(vtype=GRB.CONTINUOUS, name="x")
10      y = m.addVar(vtype=GRB.CONTINUOUS, name="y")
11      z = m.addVar(vtype=GRB.CONTINUOUS, name="z")
12
13      # Set objectives
14      m.setObjective(x, GRB.MAXIMIZE)
15
16      # Add constraints
17      m.addConstr(x + y + z == 2, "c0")
18      # Add second-order cone
19      m.addConstr(x * x + y * y <= z * z, "c1")
20
21      # Write model to file
22      m.write("SOCP.lp")
23
24      # Solve the model
25      m.optimize()
26
27      print('Optimal solution', end = "    ")
28      for i in m.getVars():
29          print('%s = %g' % (i.varName, i.x), end = " ")
30
31  except GurobiError as e:
32      print('Error code ' + str(e.errno) + ": " + str(e))
33
34  except AttributeError:
35      print('Encountered an attribute error')
```

对于规模比较大的 SOCP，可以使用 QuadExpr 来循环创建表达式，完成 SOCP 模型的建立和求解。

上述问题的求解结果如下：

```
                    Gurobi Python Interface
1  Optimal objective 9.99999978e-01
2  Optimal solution    x = 1 y = 3.07692e-09 z = 1
```

6.4 Python 调用 Gurobi 总体流程

首先给出下面这个网约车订单分配问题的模型，R 为乘客的集合，D 为司机的集合，且 $|R|=|D|$。具体模型不做详细阐述，仅从该模型作为例子。

$$\min \quad \sum_{i\in R}\sum_{j\in D} c_{ij}x_{ij} \tag{6.6}$$

$$\text{s.t.} \quad \sum_{j\in D} x_{ij}=1, \quad \forall i\in R \tag{6.7}$$

$$\sum_{i\in R} x_{ij}=1, \quad \forall j\in D \tag{6.8}$$

$$x_{ij}\in\{0,1\}, \quad \forall i\in R, \forall j\in D \tag{6.9}$$

那么如何用 Python 调用 Gurobi 来求解上述模型呢？首先，给出 Gurobi 建模求解最优化问题的一般过程。整个建模过程包含下面的步骤，如图 6.16 所示。

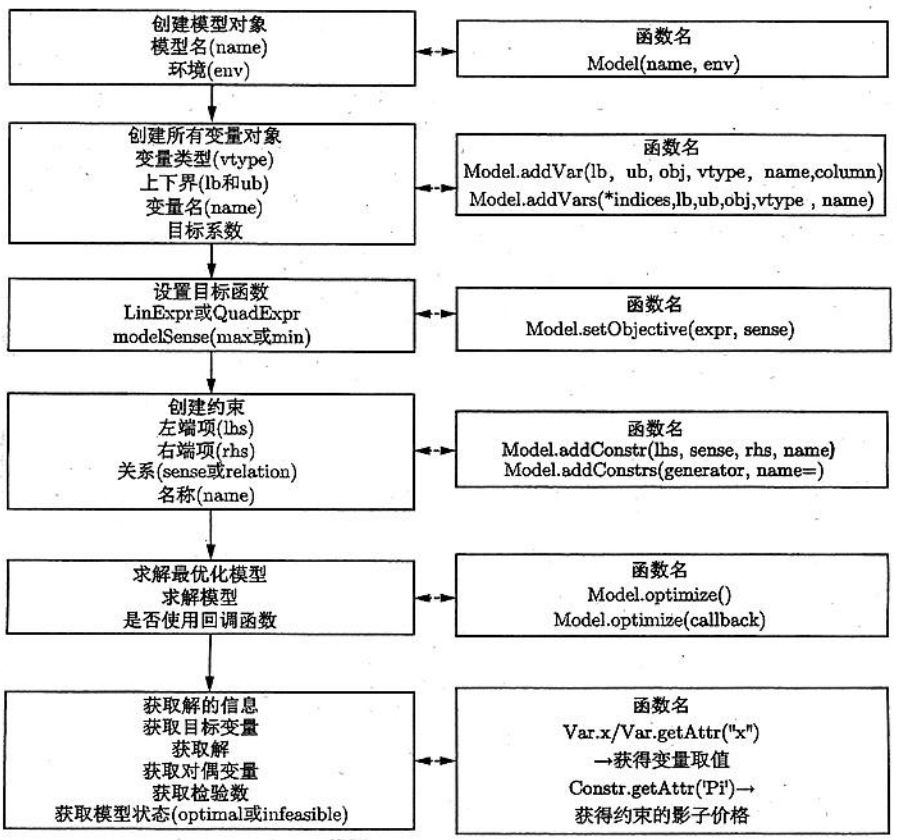


图 6.16 Gurobi 建模求解总体流程

图 6.16 很清楚地描述了 Python 调用 Gurobi 求解混合整数规划以及其他类型规划的总体步骤。

（1）创建模型对象。首先需要创建一个 Model 对象来存放该整数规划模型的所有信息，包括变量、约束等。利用函数 Model(name, env) 来创建模型对象。其中 name 是指定模型的名称，是字符串类型，而 env 是指定该模型所在的环境。使用 env 参数会使得程序更复杂，所以一般不用该参数。一般我们就用 name 参数。

（2）创建所有变量对象。创建完模型之后，需要调用 Gurobi 的 Python 接口中的 Model.addVar() 函数或者 Model.addVars() 函数，创建所有的决策变量对象，也就是 x_{ij}。创建时，我们可以对变量的取值范围（上界和下界），变量的类型（整数、0-1、连续），变量的名称（字符串）进行设定。当然有些参数是可以不设定的，此时 Gurobi 会给这些参数设置默认值。另外，Model.addVar() 函数一次只能创建一个变量，而 Model.addVars() 可以创建多个变量。Model.addVar() 和 Model.addVars() 函数做了下面的动作：首先，创建了变量对象，并将变量对象返回给接收的自定义变量，例如 x_{ij}；其次，将创建的变量添加到了已有的模型变量中。

另外，注意到 Model.addVar() 函数还有一个参数 column，这一点非常重要。我们除了按行建模的方式，还有一种按列建模的方式。在按列建模的方式中，就需要用到 column 参数。该参数一般在列生成算法等场景用得较多。

（3）设置目标函数。创建完变量，下一步就是设置目标函数。目标函数由两部分组成。第 1 部分就是要声明问题是 max 还是 min，这个在英文中称为 sense。第 2 部分就是目标函数的具体表达式。

目标函数的表达式有 2 种，一种是线性表达式（LinExpr），另一种是二次表达式（QuadExpr）。这些表达式都可以通过相应的构建方式去构建，在之后的部分会详细讲解。

构建完表达式，使用 Model.setObjective(expr, sense) 函数，给定模型名和最大化还是最小化，就可以成功设定目标函数了。

（4）创建约束。目标函数设定完成，下一步就是构建约束。约束由左端项（lhs）、符号（sense，也就是 $=, \geqslant, \leqslant$）、右端项（rhs）和约束名称（name）构成。这里需要特别注意，Gurobi 是不接受 $>$、$<$ 约束的，只接受带等号的约束。其中左端项和右端项都可以是线性表达式（LinExpr）、二次表达式（QuadExpr）或者常数这 3 种类型中的一种。

我们首先构建好线性或者二次表达式，然后利用 Model.addConstr() 函数或者 Model.addConstrs() 函数将约束添加到模型对象中去。或者说我们可以在调用 Model.addConstr() 或 Model.addConstrs() 函数时，在函数内部构建约束表达式。

（5）求解最优化模型。完成上述步骤，就可以运行 Gurobi 的求解算法了。此时，如果说我们设计了 callback 函数，我们可以将 callback 传给模型对象。如果没有的话，就可以直接用函数 Model.optimize() 来求解模型了。

（6）获取解和模型的其他信息。求解完模型，需要将最优解等信息提取出来。常用的信息包括最优解、最优值、对偶变量的值、变量的检验数（reduced cost）、模型的状态（最

优还是无解等）等。这里很多信息都属于之前创建对象的属性，比如，最优值是 Model 对象的属性，最优解和检验数是变量的属性，对偶变量的值是约束的属性等。我们需要用相应的对象，调用相应的函数去获取这些信息。

6.5 Gurobi 求解 MIP 输出的日志信息解释

6.5.1 MIP 日志示例

下面是一个 MIP 的求解日志示例：

MIP log information

```
Gurobi Optimizer version 9.0.1 build v9.0.1rc0 (win64)
Optimize a model with 8165 rows, 8268 columns and 54774 nonzeros
Model fingerprint: 0x67f12515
Variable types: 156 continuous, 8112 integer (8112 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+04]
  Objective range  [1e+00, 6e+01]
  Bounds range     [1e+00, 1e+03]
  RHS range        [1e+00, 1e+04]
Presolve removed 6588 rows and 3609 columns
Presolve time: 0.10s
Presolved: 1577 rows, 4659 columns, 21210 nonzeros
Variable types: 156 continuous, 4503 integer (4503 binary)

Root relaxation: objective 3.604485e+02, 389 iterations, 0.02 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

    0     0  360.44848    0   31          -  360.44848      -     -    0s
    0     0  360.47836    0   89          -  360.47836      -     -    0s
    0     0  361.01718    0   84          -  361.01718      -     -    0s
    0     0  361.58588    0   89          -  361.58588      -     -    0s
    0     0  361.58588    0   89          -  361.58588      -     -    0s
    0     0  361.58588    0   98          -  361.58588      -     -    0s
    0     0  361.58588    0   90          -  361.58588      -     -    0s
    0     0  361.58588    0   69          -  361.58588      -     -    0s
    0     0  361.58588    0  128          -  361.58588      -     -    0s
    0     0  361.58588    0   58          -  361.58588      -     -    0s
    0     0  361.58588    0   58          -  361.58588      -     -    0s
    0     0  361.58588    0   40          -  361.58588      -     -    0s
```

```
     0     0  361.58588    0   57          -  361.58588      -     -    0s
     0     0  361.58588    0   54          -  361.58588      -     -    0s
     0     0  361.58588    0   53          -  361.58588      -     -    1s
     0     2  362.68097    0   53          -  362.68097      -     -    1s
   711   593  486.71118   30   44          -  362.92398      -  39.5    5s
  1119   880  564.12876   76   77          -  362.97104      -  36.8   10s
  1482  1105  401.12086   31  104          -  363.96123      -  19.4   15s
* 1592  1084              95      729.1649068  363.96123  50.1%  22.2   15s
H 1618   968                      639.7685028  363.96123  43.1%  22.2   15s
H 1635   922                      639.7684334  363.96123  43.1%  22.7   15s
H 1679   920                      639.7683863  363.96123  43.1%  23.8   16s
H 1776   901                      592.6892696  364.20195  38.6%  24.9   16s
H 1826   890                      589.4353011  364.20195  38.2%  25.9   17s
  1944   909  366.00212   26  122  589.43530  364.25205  38.2%  26.3   21s
  2927  1387  559.28751   94   55  589.43530  365.01407  38.1%  33.0   26s
H 3032  1301                      567.4746422  365.01407  35.7%  33.7   26s
  4014  2169  379.11163   29   86  567.47464  366.47900  35.4%  35.2   30s
  5383  3100  464.98648   29  102  567.47464  366.96994  35.3%  35.6   35s
  7371  4744  452.71205   78   98  567.47464  368.06609  35.1%  35.5   40s
  9226  5626  475.34346   39  126  567.47464  369.05981  35.0%  36.5   46s
H 9298  5521                      554.7597868  369.13750  33.5%  36.5   46s
 10035  6131 infeasible   46        554.75979  369.55583  33.4%  38.0   50s
 10805  6509  438.51009   40   55  554.75979  370.29060  33.3%  38.2   55s
*11466  7008              68      553.4898109  370.58697  33.0%  38.3   57s
 11891  7454  485.38397   42   81  553.48981  370.99281  33.0%  38.6   60s
 13974  8716  436.92752   35   61  553.48981  372.13287  32.8%  39.2   65s
 15797  9584  390.88867   45  104  553.48981  373.22982  32.6%  40.1   70s
```

MIP 的日志信息可以分为 3 部分。

(1) 预求解部分 (presolve section)。

(2) 求解进程部分 (progress section)。

(3) 汇总部分 (summary section)。

6.5.2 预求解部分

上述日志信息的例子中，presolve section 如下：

MIP log information

```
Gurobi Optimizer version 9.0.1 build v9.0.1rc0 (win64)
Optimize a model with 8165 rows, 8268 columns and 54774 nonzeros
Model fingerprint: 0x67f12515
Variable types: 156 continuous, 8112 integer (8112 binary)
Coefficient statistics:
```

```
  Matrix range     [1e+00, 1e+04]
  Objective range  [1e+00, 6e+01]
  Bounds range     [1e+00, 1e+03]
  RHS range        [1e+00, 1e+04]
Presolve removed 6588 rows and 3609 columns
Presolve time: 0.10s
Presolved: 1577 rows, 4659 columns, 21210 nonzeros
Variable types: 156 continuous, 4503 integer (4503 binary)
```

在该例子中，presolve 部分删去了 6588 行和 3609 列，行代表约束，列代表决策变量。然后传给分支切割算法（branch-and-cut algorithm），预求解后的模型的大小发生了变化，其中含有 1577 行和 4659 列。

6.5.3 求解进程部分

Presolve 部分结束以后，就是 branch-and-cut 算法的搜索过程日志。Branch-and-cut 算法的日志信息包括了很多不同的迭代步骤和相应的信息。例如，MIPLIB 模型 mas76 例子中，首先会观察到下面的日志信息：

MIP log information

```
Found heuristic solution: objective 157344.61033
Found heuristic solution: objective 157344.61033
```

这表示 Gurobi 的启发式算法在求解根节点的线性松弛模型之前，首先通过启发式算法得到了 2 个整数可行解。

接下来就是根节点的线性松弛模型的求解日志信息。如果根节点的线性松弛模型可以很快地求解，则这个求解信息会总结成一行，如下所示：

log information

```
Root relaxation: objective 3.889390e+04, 50 iterations, 0.00 seconds
```

如果这个根节点的线性松弛模型求解比较困难，则会消耗较长时间（例如 MIPLIB 模型 dano3mip），此时 Gurobi 就会自动地将单纯形法日志信息（simplex log）显示出来。如下所示：

log information

```
Root simplex log...
Iteration Objective Primal Inf. Dual Inf. Time
15338 5.7472018e+02 6.953458e+04 0.000000e+00 5s
19787 5.7623162e+02 0.000000e+00 0.000000e+00 7s

Root relaxation: objective 5.762316e+02, 19787 iterations, 6.18 seconds
```

更准确地说，这个单纯形法日志信息在求解时间超过参数 DisplayInterval 的值之后，会自动启动（DisplayInterval 默认值为 5 秒）。

接下来的部分就是分支切割搜索树的详细过程日志信息，如下所示：

MIP log information

	Nodes		Current Node			Objective Bounds			Work	
	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
	0	0	360.44848	0	31	-	360.44848	-	-	0s
	0	0	360.47836	0	89	-	360.47836	-	-	0s
	0	0	361.01718	0	84	-	361.01718	-	-	0s
	0	0	361.58588	0	89	-	361.58588	-	-	0s
	0	0	361.58588	0	89	-	361.58588	-	-	0s
	0	0	361.58588	0	98	-	361.58588	-	-	0s
	0	0	361.58588	0	90	-	361.58588	-	-	0s
	0	0	361.58588	0	69	-	361.58588	-	-	0s
	0	0	361.58588	0	128	-	361.58588	-	-	0s
	0	0	361.58588	0	58	-	361.58588	-	-	0s
	0	0	361.58588	0	58	-	361.58588	-	-	0s
	0	0	361.58588	0	40	-	361.58588	-	-	0s
	0	0	361.58588	0	57	-	361.58588	-	-	0s
	0	0	361.58588	0	54	-	361.58588	-	-	0s
	0	0	361.58588	0	53	-	361.58588	-	-	1s
	0	2	362.68097	0	53	-	362.68097	-	-	1s
	711	593	486.71118	30	44	-	362.92398	-	39.5	5s
	1119	880	564.12876	76	77	-	362.97104	-	36.8	10s
	1482	1105	401.12086	31	104	-	363.96123	-	19.4	15s
*	1592	1084		95		729.1649068	363.96123	50.1%	22.2	15s
H	1618	968				639.7685028	363.96123	43.1%	22.2	15s
H	1635	922				639.7684334	363.96123	43.1%	22.7	15s
H	1679	920				639.7683863	363.96123	43.1%	23.8	16s
H	1776	901				592.6892696	364.20195	38.6%	24.9	16s
H	1826	890				589.4353011	364.20195	38.2%	25.9	17s
	1944	909	366.00212	26	122	589.43530	364.25205	38.2%	26.3	21s
	2927	1387	559.28751	94	55	589.43530	365.01407	38.1%	33.0	26s
H	3032	1301				567.4746422	365.01407	35.7%	33.7	26s
	4014	2169	379.11163	29	86	567.47464	366.47900	35.4%	35.2	30s
	5383	3100	464.98648	29	102	567.47464	366.96994	35.3%	35.6	35s
	7371	4744	452.71205	78	98	567.47464	368.06609	35.1%	35.5	40s
	9226	5626	475.34346	39	126	567.47464	369.05981	35.0%	36.5	46s
H	9298	5521				554.7597868	369.13750	33.5%	36.5	46s
	10035	6131	infeasible	46		554.75979	369.55583	33.4%	38.0	50s
	10805	6509	438.51009	40	55	554.75979	370.29060	33.3%	38.2	55s

```
39 *11466   7008              68      553.4898109   370.58697   33.0%   38.3    57s
40  11891   7454   485.38397   42    81  553.48981   370.99281   33.0%   38.6    60s
41  13974   8716   436.92752   35    61  553.48981   372.13287   32.8%   39.2    65s
42  15797   9584   390.88867   45   104  553.48981   373.22982   32.6%   40.1    70s
43  ...........................................................................
44  ...........................................................................
```

MIP 的 branch-and-cut 求解日志信息会非常多，但是每一列的信息都比较清楚地反映了求解的进度。

（1）Nodes 部分：包括前 2 列 Expl 和 Unexpl。第 1 列 Expl 是分支切割树已经探索到的节点（也就是已经探明的节点，或者说已经被剪枝的节点）。第 2 列 Unexpl 表示还没被探明的叶子节点（leaf nodes），也就是还可以继续分支的节点。

注意：有时会在日志信息开头有 H 或者 * 的字样，其中 H 表示 Gurobi 内部的启发式找到了一个可行解，* 表示由分支操作得到了一个可行解。

（2）Current Node 部分：该部分提供了分支切割树中当前正在探索的节点的信息。Obj 是当前节点线性松弛模型的目标函数。Depth 表示当前节点在分支切割树中的深度。IntInf 表示在当前节点的线性松弛模型的解中，整数决策变量取了小数值的变量的个数。

（3）Objective Bounds 部分：该部分中 Incumbent 展示的是到目前为止已经获得的最好的可行解（也就是整数解）的目标函数（也就是当前最优解 current incumbent 对应的目标函数）。BestBd 是分支切割算法搜索树中的叶子节点（leaf nodes）提供的目标函数的当前最好界限（Bound）。MIP 的最优值一定介于 Incumbent 和 BestBd 之间。Gap 指的是 Incumbent 和 BestBd 之间的相对间隙（Gap）。即

$$\text{Gap} = \frac{\text{Incumbent} - \text{BestBd}}{\text{Incumbent}} \times 100\%$$

注意：Gap 是以当前最好的整数可行解（incumbent）的值为基准计算的相对间隙。当间隙比参数 MIPGap 的值小时，算法终止。按照 Gurobi 中 Gap 的计算公式，Gap 可能为负值。不同求解器中，Gap 的计算方法不同。

（4）Work 部分：该部分展示了到该节点处所进行的工作量信息。第 1 列 It/Node 表示在分支切割树中，平均每个节点处单纯形法迭代的次数。Time 是到目前为止分支切割算法执行的累计时间。

6.5.4 汇总部分

第 3 部分的日志信息就是对 MIP 求解器已经完成的所有求解过程的汇总信息，如下所示：

log information

```
1 Cutting planes:
2         Gomory: 2
```

```
        MIR: 14
Explored 241338 nodes (1336406 simplex iterations) in 6.57 seconds
Thread count was 4 (of 4 available processors)

Solution count 7: 40005.1 40697.1 41203.6 ... 157345
Optimal solution found (tolerance 1.00e-04)
Best objective 4.000505414200e+04, best bound 4.000505414200e+04, gap
      0.0000%
```

在本例中，只用了 6.57 秒就得到了最优状态。且有 tolerance 1.00e-04 的字样，因此最优整数可行解与最优界限（bound）之间的 gap 小于 0.01%。并且程序最终的求解状态为 Optimal，因为最终的间隙为 0.0000%，小于 MIPGap 的值。

6.6 Python 接口概述

6.6.1 模型概述

Gurobi 的 Python 接口中的大多数动作都可以通过直接调用 Gurobi 中的对象实现。最常用的对象就是 Model 对象。一个模型包含一系列决策变量（Var 或者 MVar 类的对象）、一个关于这些变量的线性或者二次的目标函数（用 Model.setObjective 设置）、一系列关于这些变量的约束（Constr，QConstr，SOS，GenConstr 类的对象），每一个变量有对应的上界、下界、变量类型（连续、0-1、整数）、名称。每一个线性约束或者二次约束都有相应的符号（$\geqslant$、$\leqslant$、$=$）和右端项。

一个模型对象可以一次性通过读入模型文件创建。也可以首先创建一个空的 Model 对象，然后调用 Model.addVar，Model.addVars，Model.addMVar 去逐渐添加变量，然后调用 Model.addConstr，Model.addConstrs，Model.addLConstr，Model.addQConstr，Model.addSOS 或者任意一个 Model.addGenConstrXxx 函数来添加约束。

线性约束可以通过创建线性表达式来添加。首先创建线性表达式对象（class LinExpr 或者 MLinExpr 的对象），然后声明这些线性表达式之间的关系（$\geqslant$、$=$、$\leqslant$）。二次约束也采用相似的方法，但是需要创建二次表达式（class QuadExpr 或者 MQuadExpr 的对象）。广义约束（General Constraints）需要创建一系列的专用函数（dedicated methods），或者一系列的广义约束帮助函数（general constraint helper functions）和相应的重载运算符（overloaded operators）。

模型（class Model）对象是动态的实体，可以对其动态地增加或者删除决策变量或者约束。一个模型（model），如果目标函数是线性的，且约束是线性的，所有决策变量都是连续型的，该模型是线性规划（Linear Program，LP）。如果目标函数的二次的，则该模型是一个二次规划（Quadratic Program，QP）。如果模型中任意约束是二次的，则该模型是一个二次约束规划（Quadratically-Constrained Program，QCP）。有时 QCP 中有一些特例：约束为凸的 QCP，约束为非凸的 QCP，bilinear 规划（二次项都是 0-1 变量相乘）和二阶锥规划

(Second-Order Cone Program，SOCP)。如果模型中任意变量是整数变量或半连续变量、半整数变量，或者 Special Ordered Set（SOS）约束或某些广义约束（General Constraints），则该模型为混合整数规划（Mixed Integer Programming，MIP）。MIP 中又包含一些特例，包括混合整数线性规划（Mixed Integer Linear Programming，MILP），混合整数二次规划（Mixed Integer Quadratic Programming，MIQP），混合整数二次约束规划（Mixed Integer Quadratically-Constrained Programming，MIQCP）和混合整数二阶锥规划（Mixed Integer Second-Order Cone Programming，MISOCP）。对这些类别的模型，Gurobi 都可以求解。

6.6.2 求解模型

在创建完模型以后，可以调用 `Model.optimize` 函数求解模型。默认情况下，`optimize` 函数将会使用并发求解器（Concurrent Optimizer）求解线性规划模型，用内点法（the barrier algorithm）求解目标函数为凸的二次规划（QP）和约束为凸的二次约束规划（QCP），对于其余情况下的模型均使用 branch-and-cut 算法进行求解。

Gurobi 求解器将会把模型的求解状态都存储在模型对象内，因此调用求解函数 `Model.optimize` 求解模型之后，模型的求解状态等相关信息也会发生相应的变化。如果想要清除之前求解的结果信息，而不对模型的约束和变量等信息做任何改变，然后重新对模型进行求解，则可以调用 `Model.reset` 函数重置模型对象。

求解完一个 MIP 模型之后，我们可以调用函数 `Model.fixed` 计算相应的固定的模型（fixed model）。这个被固定的模型和原来的模型是相同的，只是所有的整数变量都被固定成了 MIP 模型的解中的值。如果模型中包含 SOS 约束，此时一些出现在 SOS 约束中的连续变量也有可能被固定。有时这些固定操作会有助于计算一些被固定的模型的信息（如对偶变量的值、敏感性分析的信息等）。

6.6.3 多个解、目标函数和场景

默认情况下，Gurobi 优化器假设用户的目的就是为了找到一个具有单一目标函数的模型的最优解。但是 Gurobi 也提供了下面的特征，以便用户来修改这些假设。

（1）`Solution Pool`：允许用户找到更多的解。

（2）`Multiple Scenarios`：允许用户找到多个相关模型的解。

（3）`Multiple Objectives`：允许用户声明多个目标函数，并且控制这些目标函数之间的权衡。

6.6.4 不可行的模型

如果一个模型是不可行的，用户也可以进行一些相应的操作。此时用户可以尝试去诊断导致模型不可行的原因，或者试图去修复模型的不可行性。为了得到模型不可行的原因的诊断信息，可以调用函数 `Model.computeIIS` 计算不可约不相容子方程组（Irreducible Inconsistent Subsystem，IIS）。该方法既可以用于线性规划模型，也可以用于混合整数规划模型，当然，混合整数规划模型将会花费较长的时间。该方法将会产生一系列的 IIS 属性。

如果想尝试去修复不可行性，我们可以调用 Model.feasRelaxS 函数或者 Model.feasRelax 函数计算一个原模型的可行的松弛版本模型。这个松弛的模型会得到一个约束违背程度最小的解。

6.6.5 查询和修改模型属性

大多数与 Gurobi 模型相关的信息都存储在一组属性中。有些属性与模型的变量相关联，有些与模型的约束相关，还有一些与模型本身有关。举一个简单的例子，求解模型会生成变量的 x 属性（也就是变量在解中的对应取值）。由 Gurobi 计算的，像 x 这样的属性，用户是无法直接修改的，而其他的属性，例如变量的上界（ub 属性），则可以直接被用户修改。

在 Python 接口中，属性可以通过 2 种方法获得。第 1 种就是调用 getAttr() 和 setAttr() 方法，更具体地说，这些方法可以用于获得变量的属性（Var.getAttr/Var.setAttr）、线性约束的属性（Constr.getAttr/Constr.setAttr）、二次约束的属性（QConstr.getAttr/QConstr.setAttr）、SOS 的属性（SOS.getAttr）、广义约束的属性（GenConstr.getAttr/GenConstr.setAttr）和模型的属性（Model.getAttr/Model.setAttr）。调用这些函数时，都是将属性名作为第一个传入参数的，例如 var.getAttr("x") 和 constr.setAttr("rhs", 0.0)。全部的属性名见 Gurobi 用户手册 Attributes 部分。

属性也可以通过更直接的方法获得。也就是对象后跟属性名就可以直接获得。例如 b = constr.rhs 和 b = constr.getAttr("rhs") 就是等价的。类似地，constr.rhs = 0.0 和 constr.setAttr("rhs", 0.0) 也是等价的。

注意：属性名可以忽略大小写。比如 model.objval 和 model.ObjVal 都是可以的。

6.6.6 其他修改模型信息的方法

大多数修改现有模型的操作都可以通过属性名接口进行修改（如修改变量的上下界、约束的右端项等）。但是也有一些特例，例如修改约束矩阵和改变目标函数等，就不能通过属性接口来修改。

约束矩阵可以通过多种方法来修改。第 1 种就是调用 Model.chgCoeff 函数。该函数可用于修改一个已经存在的非零系数的值，或者设置一个已经存在的非零系数为 0，或者创建一个新的非零系数。约束矩阵同样也会随着变量的移除、约束的移除而改变。当用户调用 Model.remove 函数移除变量或者约束时，约束矩阵会随之自动改变。

目标函数也可以通过其他方法来修改。最开始的时候，用户可以通过建立表达式来创建目标函数（LinExpr , MLinExpr, QuadExpr 或者 MQuadExpr 对象），然后将其传给设置目标函数的函数 Model.setObjective 即可完成目标函数的设置。如果用户想修改目标函数，可以通过重新创建新的 LinExpr 或者 QuadExpr 对象，然后再次调用 Model.setObjective 函数即可。

当然，对于线性的目标函数而言，也可以在调用 Model.addVar() 函数创建变量时，设置其中的参数 Obj 的值来改变目标函数系数，或者通过修改变量 Var 的 Obj 属性来修改

目标函数。

如果目标函数是分段线性的，用户可以调用 Model.setPWLObj 函数来声明。对每一个相关的决策变量都调用一次该函数即可。

Gurobi 的单纯形求解器包含了对凸的分段线性目标函数的算法支持，因此对于连续型模型用此功能，将会有非常好的性能优势。如果要清除先前指定的分段线性目标函数，只需设置相应变量的 Obj 属性为 0 即可。

6.6.7 惰性更新

惰性更新是关于模型修改的一个概念。也就是以惰性的形式来对模型进行修改，这些惰性的修改不会立即对模型产生影响，而是首先将它们放入一个队列，等待之后再被运行生效。简单地创建模型并求解，是不会涉及这个部分的，但是如果用户想在模型被修改之前，获取未被修改的模型的信息，那就需要知道惰性更新的事情。

模型的修改信息（如变量上下界的修改、约束右端项的修改、目标函数的修改等）会被放在一个队列中，这个队列中的修改会在下面 3 种情况下被真正执行。第 1 种情况就是直接明了地调用函数 Model.update。第 2 种就是调用求解函数 Model.optimize。第 3 种就是调用函数 Model.write 将模型导出成本地模型文件。第一种方法是非常细微精准的操作，可以随时根据需要更新模型。后两种是用户在求解模型之前，想把所有的修改信息一并执行而进行的操作。

为什么 Gurobi 接口会以这种方式运行？有几个原因。首先是这种方法使对模型进行多次修改变得容易得多，因为在不同的模型修改操作之间，模型是保持不变的。二是处理模型修改信息可能是很耗时间的，尤其是在处理服务器环境中机器之间的通信问题时。通常，如果用户的程序需要进行多次修改，用户可以分阶段进行修改，首先进行一组修改，然后更新，然后进行更多修改，然后再次更新，等等。每次单个的修改操作都去调用一次 Model.update 可能会很耗时。

如果用户忘记了调用更新函数，用户的程序也不会崩溃。程序会返回最近一次更新后的模型的信息。如果模型对象已经不存在了，程序会返回一个 NOT_IN_MODEL 异常。

6.6.8 参数管理

Gurobi 提供了非常多的参数来满足用户控制求解进程的需求。这些控制因素包括可行性、最优容差、算法选择、MIP 搜索树搜索策略等。这些参数都可以在求解模型之前通过相应的方法进行修改。这些参数都可以通过调用函数 Model.setParam 进行修改。模型当前的参数可以通过函数 Model.getParamInfo 获得，用户也可以通过 Model.Params 类更直接地获得参数的值。例如，用户需要设置模型 m 的 MIPGap，可以用 m.setParam('MIPGap', 0) 或者 m.Params.MIPGap=0。

用户也可以通过调用 Model.read 函数，从文件中读取一系列的参数设置信息，或者调用 Model.write 函数将修改好的参数信息写入一个文件中。

用户也可以用参数调优工具来自动探索不同参数组合，以提升求解的效率。可以调用函数 Model.tune 去激活模型中的调优工具。具体信息可以看 Gurobi 用户手册中的 parameter tuning tool 部分。

另外需要注意的是，用户对一个模型的参数修改操作不会影响另一个模型的参数。如果想要对所有的模型都进行统一的参数修改操作，可以用全局 setParam 方法实现。

模型的参数列表见 Gurobi 用户手册的 Parameters 部分。

6.6.9 管理求解进程：日志和回调

用户可以通过 Gurobi 的日志来管理 Gurobi 的优化进程。默认情况下，Gurobi 会将日志打印在屏幕上。但是也有一些修改日志行为的控制接口。用户可以通过设置 LogFile 参数来导出日志文件。日志文件的打印频率可以通过修改参数 DisplayInterval 来控制。如果想要关闭日志文件的显示，可以设置输出参数 OutputFlag 的值为 0。

更详细的日志信息可以通过 callback 函数获得。如果用户为求解函数 Model.optimize 传入一个 callback 函数，参数为 model 和 where，这个 callback 函数将会周期性的被优化程序调用。用户的 callback 函数之后会调用 Model.cbGet 以获取优化程序的额外信息。

6.6.10 修改求解器的行为：回调

函数 callback 也可以修改求解器的行为。最简单的控制 callback 是 Model.terminate，它要求优化器在最早的方便的时间点终止程序。函数 Model.cbSetSolution 可以用来在 MIP 的求解过程中为模型设置一个可行解（或者解的一部分）。函数 Model.cbCut 和 Model.cbLazy 可以用来在 MIP 求解过程中添加割平面和惰性约束（lazy constraint）。

6.7 Python 调用 Gurobi 常用类和函数

6.7.1 全局函数

Gurobi 的 Python 接口的部分全局函数如表 6.1 所示。

表 6.1 全局函数

函 数 名	功 能
paramHelp (paramname)	获得参数的帮助信息
quicksum (data)	对列表中的元素进行快速求和
read (filename)	从文件读入模型
readParams (filename)	从文件读入参数设置
setParam (paramname)	设置参数的值
writeParams (filename)	将模型的参数设置写成文件